



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：动态流水线设计与性能定量分析

学号：2051454

姓名：王逸菲

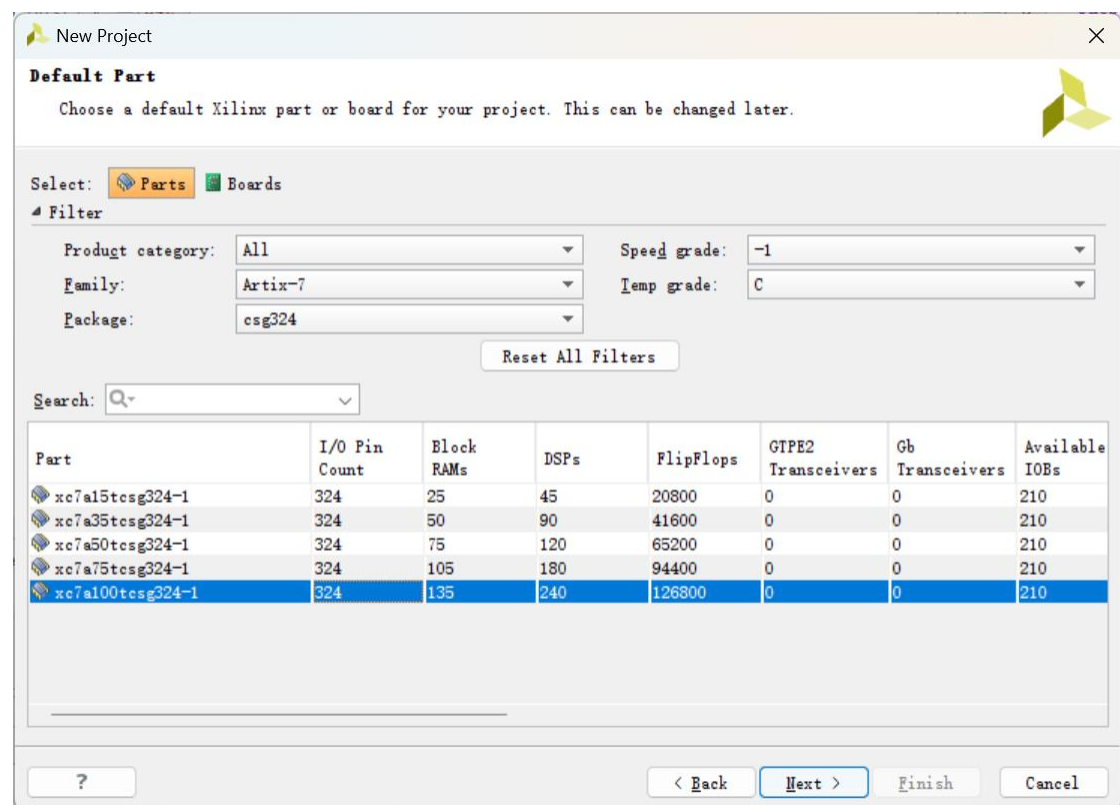
指导教师：陆有军

日期：2023 年 12 月 23 日

一、实验环境部署与硬件配置说明

1、实验环境部署

在 Vivado 中创建下面配置的项目：



2、硬件配置说明

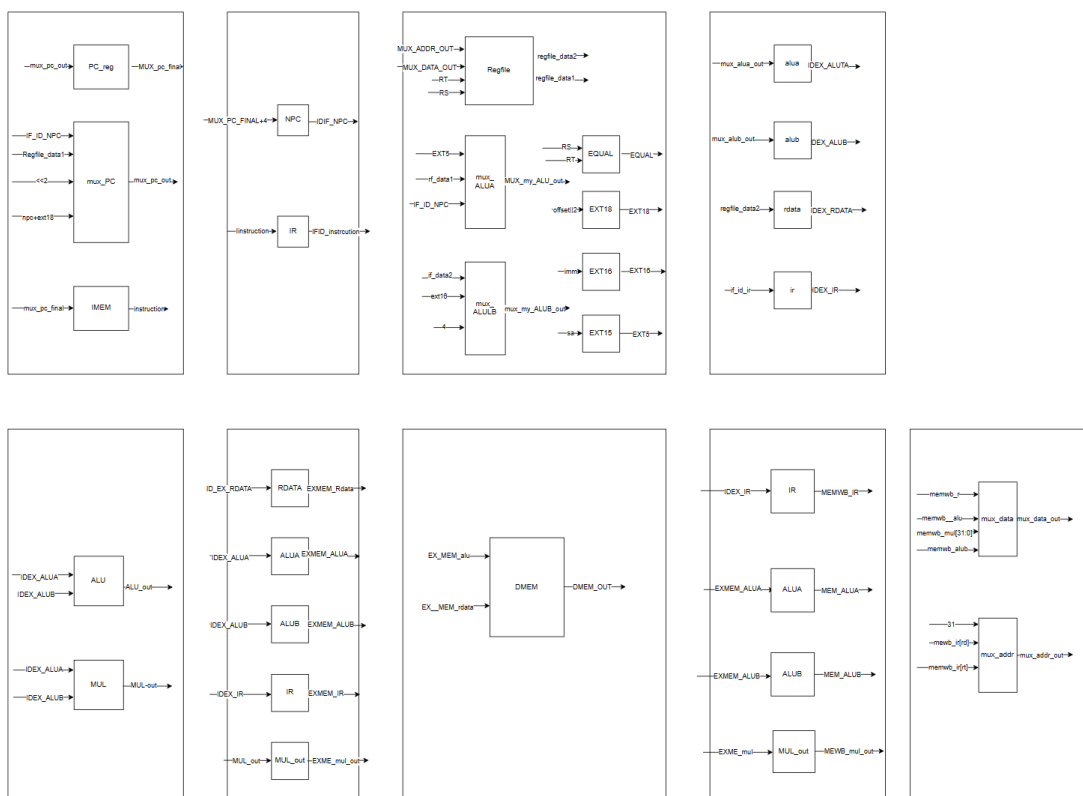
操作系统：windows11

软件：Vivado 2016.2

开发板：XILINX NEXYS4

二、实验的总体结构

1、动态流水线的总体结构



2、指令说明：

本实验实现 MIPS31 条指令，实现 31 条指令的动态流水线执行。

具体地，实现了以下 31 条指令：

Mnemonic Symbol	Format						Sample
Bit #	31..26	25..21	20..16	15..11	10..6	5..0	
R-type	op	rs	rt	rd	shamt	func	
add	000000	rs	rt	rd	0	100000	add \$1,\$2,\$3
addu	000000	rs	rt	rd	0	100001	addu \$1,\$2,\$3
sub	000000	rs	rt	rd	0	100010	sub \$1,\$2,\$3
subu	000000	rs	rt	rd	0	100011	subu \$1,\$2,\$3
and	000000	rs	rt	rd	0	100100	and \$1,\$2,\$3
or	000000	rs	rt	rd	0	100101	or \$1,\$2,\$3
xor	000000	rs	rt	rd	0	100110	xor \$1,\$2,\$3
nor	000000	rs	rt	rd	0	100111	nor \$1,\$2,\$3
slt	000000	rs	rt	rd	0	101010	slt \$1,\$2,\$3
sltu	000000	rs	rt	rd	0	101011	sltu \$1,\$2,\$3
sll	000000	0	rt	rd	shamt	000000	sll \$1,\$2,\$3
srl	000000	0	rt	rd	shamt	000010	srl \$1,\$2,\$3
sra	000000	0	rt	rd	shamt	000011	sra \$1,\$2,\$3
slv	000000	rs	rt	rd	0	000100	slv \$1,\$2,\$3
sriv	000000	rs	rt	rd	0	000110	sriv \$1,\$2,\$3
srav	000000	rs	rt	rd	0	000111	srav \$1,\$2,\$3
jr	000000	rs	0	0	0	001000	jr \$31

Bit #	31..26	25..21	20..16	15..0	
I-type	op	rs	rt	immediate	
addi	001000	rs	rt	Immediate(- ~ +)	addi \$1,\$2,100
addiu	001001	rs	rt	Immediate(- ~ +)	addiu \$1,\$2,100
andi	001100	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
ori	001101	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
xori	001110	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
lw	100011	rs	rt	Immediate(- ~ +)	lw \$1,10(\$2)
sw	101011	rs	rt	Immediate(- ~ +)	sw \$1,10(\$2)
beq	000100	rs	rt	Immediate(- ~ +)	beq \$1,\$2,10
bne	000101	rs	rt	Immediate(- ~ +)	bne \$1,\$2,10
slti	001010	rs	rt	Immediate(- ~ +)	slti \$1,\$2,10
sltiu	001011	rs	rt	Immediate(- ~ +)	sltiu \$1,\$2,10
lui	001111	00000	rt	Immediate(- ~ +)	Lui \$1, 10
Bit #	31..26	25..0			
J-type	op	Index			
j	000010	address			j 10000
jal	000011	address			jal 10000

下面是 31 条指令的逐条指令分析：

1) ADD

格式：add rd, rs, rt

操作：PC→ IMEM（取指）、rs+rt→rd、NPC (PC + 4) → PC

部件：PC、NPC、IMEN、RegFiles、ALU

2) ADDU（unsigned 没有溢出）

格式：add rd, rs, rt

操作：PC→ IMEM（取指）、rs+rt→rd、NPC (PC + 4) → PC

部件：PC、NPC、IMEN、RegFiles、ALU

3) SUB

格式：sub rd, rs, rt

操作：PC→ IMEN（取指）、rs-rt→rd、NPC (PC+4) → PC

部件：PC、NPC、IMEN、RegFiles、ALU

4) SUBU（unsigned 没有溢出）

格式：sub rd, rs, rt

操作：PC→ IMEN（取指）、rs-rt→rd、NPC (PC+4) → PC

部件：PC、NPC、IMEN、RegFiles、ALU

5) AND

格式：and rd, rs, rt

操作：PC→ IMEN（取指）、rs & rt→rd、NPC (PC+4) → PC

部件：PC、NPC、IMEN、RegFiles、ALU

6) OR

格式: or rd, rs, rt

操作: PC → IMEN (取指)、rs or rt → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

7) XOR (异或)

格式: xor rd, rs, rt

操作: PC → IMEN (取指)、rs xor rt → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

8) NOR (按位或并取反)

格式: nor rd, rs, rt

操作: PC → IMEN (取指)、rs nor rt → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

9) SLT (有符号数比较: 前者小于后者返回 1, 前者大于后者返回 0)

格式: slt rd, rs, rt

操作: PC → IMEN (取指)、rs < rt → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU、EXT1

10) SLTU (无符号数比较: 前者小于后者返回 1, 前者大于后者返回 0)

格式: sltu rd, rs, rt

操作: PC → IMEN (取指)、rs < rt → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU、EXT1

11) SLL (sa 原是 5 位, 无符号扩展为 32 位, rs 逻辑左移 sa 位, 空余位补 0)

格式: sll rd, rs, sa

操作: PC → IMEN (取指)、IMEM[10:6](sa) → EXT5、rs << sa(logical) → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、Ext5、ALU

12) SRL (sa 原是 5 位, 无符号扩展为 32 位, rs 逻辑右移 sa 位, 空余位补 0)

格式: srl rd, rs, sa

操作: PC → IMEN (取指)、IMEM[10:6](sa) → EXT5、rs >> sa(logical) → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、Ext5、ALU

13) SRA (sa 原本是 5 位, 无符号扩展为 32 位, rs 算数右移 sa 位, 空余位补符号位)

格式: sra rd, rs, sa

操作: PC → IMEN (取指)、IMEM[10:6](sa) → EXT5、rs >> sa(arithmetic) → rd、

NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、Ext5、ALU

14) SLLV (两个操作数均从寄存器中出, 移位位数由地址为 rs 的寄存器值的第 0-4bit 确定)

格式: sllv rd, rt, rs

操作: PC → IMEN (取指)、 $rt \ll rs[4:0]$ (logical) → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

15) SRLV (两个操作数从寄存器中出, 移位位数由地址为 rs 的寄存器值的第 0-4bit 确定)

格式: srlv rd, rt, rs

操作: PC → IMEN (取指)、 $rt \gg rs[4:0]$ (logical) → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

16) SRAV (两个操作数从寄存器中出, 算数右移)

格式: srav rd, rt, rs

操作: PC → IMEN (取指)、 $rt \gg rs[4:0]$ (arithmetic) → rd、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles、ALU

17) JR (无条件转移指令, 转移到 rs 保存的地址中)

格式: jr rs

操作: PC → IMEN (取指)、rs → PC、NPC (PC+4) → PC

部件: PC、NPC、IMEN、RegFiles

18) ADDI

格式: addi rt, rs, immediate

操作: PC → IMEN (取指)、 $rs + \text{imm16}$ (有符号扩展) → rt、NPC (PC+4) → PC

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

19) ADDIU

格式: addiu rt, rs, immediate

操作: PC → IMEN (取指)、 $rs + \text{imm16}$ (有符号扩展) → rt、NPC (PC+4) → PC

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

20) ANDI

格式: andi rt, rs, immediate

操作: PC → IMEN (取指)、 $rs \text{ and } \text{imm16}$ (有符号扩展) → rt、NPC (PC+4) → PC

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

21) ORI

格式: ori rt, rs, immediate

操作: PC → IMEN (取指)、 $rs \text{ or } \text{imm16}$ (有符号扩展) → rt、NPC (PC+4) → PC

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

22) XORI

格式: `xori rt, rs, immediate`

操作: $PC \rightarrow IMEN$ (取指)、 $rs \text{ xor } imm16$ (有符号扩展) $\rightarrow rt$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

23) LW (读内存到 rd)

格式: `lw rt offset(base)`

操作: $PC \rightarrow IMEN$ (取指)、 $memory[rs+offset] \rightarrow rt$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU、DMEM

24) SW (存数据 rt)

格式: `sw rt offset(base)`

操作: $PC \rightarrow IMEN$ (取指)、 $rt \rightarrow memory[rs+offset]$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU、DMEM

25) BEQ

格式: `beq rs, rt, offset`

操作: $PC \rightarrow IMEN$ (取指)、 $\text{if } (rs == rt) NPC + Sign_ext(Offset || 02) \text{ else } NPC(PC+4) \rightarrow PC$

Offset || 02 表示将 offset 左移 2 位

部件: PC、NPC、IMEN、EXT18、RegFiles、ALU、ADD

26) BNE

格式: `bne rs, rt, offset`

操作: $PC \rightarrow IMEN$ (取指)、 $\text{if } (rs != rt) NPC + Sign_ext(Offset || 02) \text{ else } NPC(PC+4) \rightarrow PC$

Offset || 02 表示将 offset 左移 2 位

部件: PC、NPC、IMEN、EXT18、RegFiles、ALU、ADD

27) SLTI (rs 小于立即数, rt 就赋 1)

格式: `slti rt, rs, immediate`

操作: $PC \rightarrow IMEN$ (取指)、 $(rs < \text{ext.imm16}(\text{有符号扩展})) \rightarrow rt$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU、EXT1

28) SLTIU (无符号比较, rs 小于立即数, rt 就赋 1)

格式: `slti rt, rs, immediate`

操作: $PC \rightarrow IMEN$ (取指)、 $(rs < \text{ext.imm16}(\text{有符号扩展})) \rightarrow rt$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU、EXT1

29) LUI (将立即数左移 16 位, 存入 rt 中)

格式: `lui rt, immediate`

操作: $PC \rightarrow IMEN$ (取指)、 $\text{imm16} || 016 \rightarrow rt$ 、 $NPC(PC+4) \rightarrow PC$

部件: PC、NPC、IMEN、EXT16、RegFiles、ALU

30) J (直接跳转到立即数表示的地址中)

格式: j target

操作: PC → IMEN (取指)、PC31-28 || instr_index || 02、NPC(PC+4) → PC

部件: PC、NPC、IMEN、||

31) JAL (Jump And Link, 需要原地址 pc+8 (跳转后的下一个地址) 到 rd 中)

格式: jal target

操作: PC → IMEN (取指)、(rs < ext.imm16 (有符号扩展)) → rt

部件: PC、NPC、IMEN、||、Regfiles、ADD8、

3、动态流水线的结构

动态流水线分为 IF 段, ID 段, EX 段, MEM 段, WB 段。

由于需要解决由写后读以及指令跳转指令引发的流水线暂停 stall, 所以在上述五段中间增设 IF/ID 段, ID/EX 段, EX/MEM 段, MEM/WB 段, 用于段间的数据传递, 用数据断流来实现可能的流水暂停。例如: IF/ID 段: IF 段到 ID 段的数据传递段。该段保存 IF 段生成的指令, 并将其传递给 ID 段。在这里, 可以进行一些前期处理, 例如处理指令跳转。通过在这些段之间插入中间寄存器和断流逻辑, 可以实现在写后读和指令跳转引发流水线暂停时的数据处理。这种流水线设计方式有助于提高流水线的效率和性能, 使得计算机能够更有效地执行指令序列。

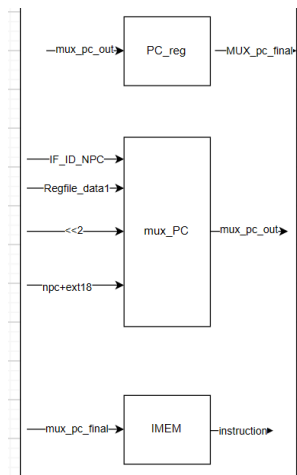
三、总体架构部件的解释说明

1、动态流水线总体结构部件的解释说明

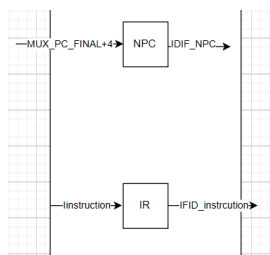
1、IF 段

IF 段中 mux_pc 部件对指令的地址进行选择, 将结果输入到 pc_reg 中, 当指令为 bne 或者 beq 时, 需要在后面的 equal 部件中判断是否满足条件, 如果满足条件, 就跳转到 ext18 那一项中, 如果指令时 jr, 就选择 rf_data1, 如果指令时 j 时就选择左移那一项, 其余情况就选 npc。

pc 模块的输出作为 IMEM 取指模块的输入。



2、IF/ID 段进行数据的传递。在此不再赘述。



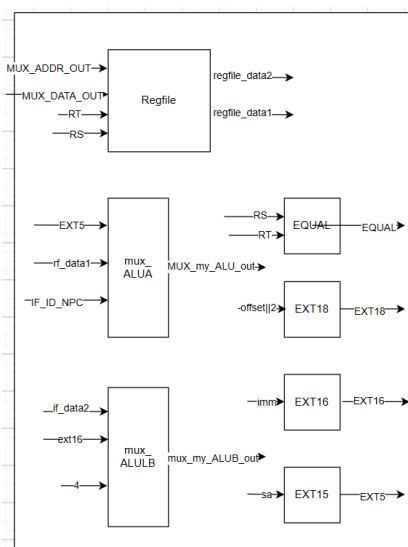
3、ID 段：

RegFile 模块实现寄存器组，它接收 Rs 寄存器地址、Rt 寄存器地址、要写入的寄存器地址以及要写入的数据作为输入。模块输出包括读出的寄存器 Rs 的值（regfile_data1）和读出的寄存器 Rt 的值（regfile_data2）。

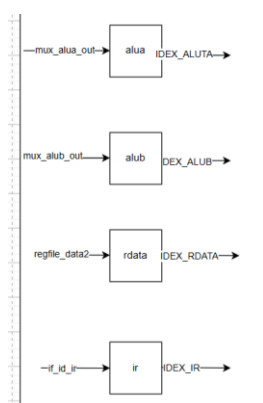
ID 段提前判断跳转指令，输入为 Rs 和 Rt，输出比较结果。

EXT18,EXT16,EXT5 分别是根据指令进行符号或者零扩展。

Mux_alua, mux_alub 使用三路选择器，用于选择后面 ALU 的两个操作数。

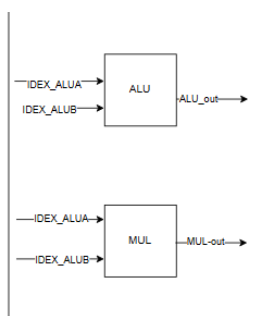


4、ID/EX 段进行数据的传递。

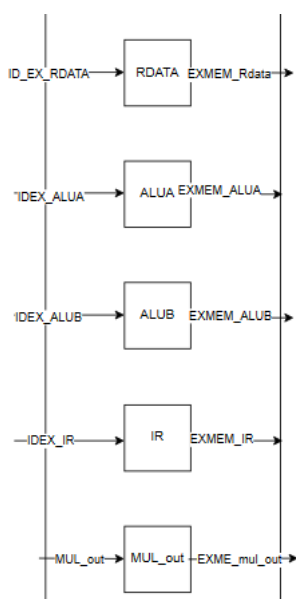


5、EX 段

在 31+2 条指令集中涉及到的运算由 alu 这个算术逻辑单元就可以全部解决，可以实现所有指令的运算，运算结果由 ALU_out 输出。MUL 这个有符号乘法器解决 mul 指令的运算，结果由 MUL_out 输出。

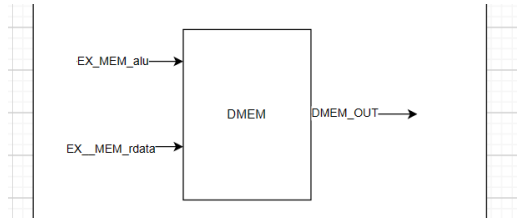


6、EX/MEM 段用于数据的传递。

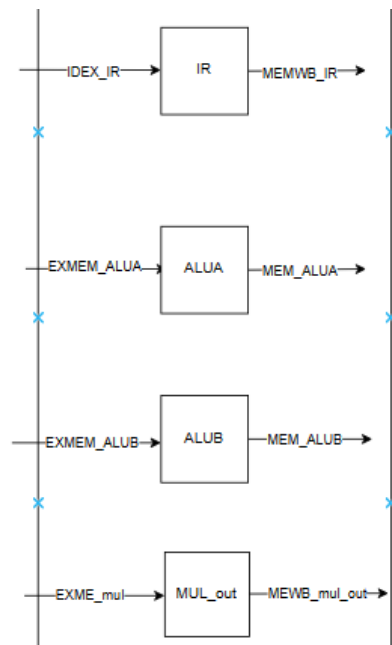


7、MEM 段

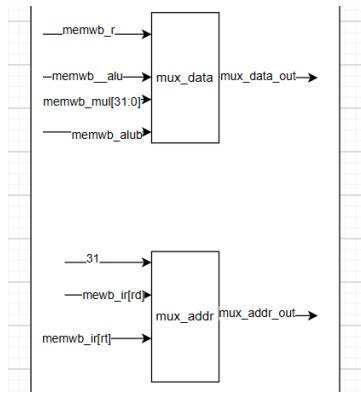
对内存的读取和写入操作阶段，只有涉及到访存的指令会用到该阶段。输入要访问或者写入的地址，输出就是如果是取数指令的话，取到的数据。



8、MEM/WB 段进行数据的传递。



9、WB 段负责数据的写回，主要操作涉及对 RegFile 的操作。此段处理 data_out 和 addr_out 两个输出，将数据写回到 RegFile 中。具体来说，该段通过指定写回的数据（data_out）和地址（addr_out），与 ID 段的寄存器进行配合，实现对 RegFile 的写回操作。



四、实验仿真过程

1、动态流水线的前仿真过程测试代码

```

1. module test;
2.   reg clk;
3.   reg rst;
4.   wire [31:0] inst, pc;
5.   integer file_open;
6.   reg [31:0] cnt;
7.
8.   initial begin
9.     file_open = $fopen("output.txt");
10.    clk = 1'b0;
11.    rst = 1'b1;
12.    cnt = 0;
13.    #10 rst = 1'b0;
14.  end
15.
16.  always begin
17.    #3 clk <= !clk;
18.  end
19.
20.  always @(posedge clk) begin
21.    if (!rst) begin
22.      cnt <= cnt + 1'b1;
23.      $fdisplay(file_open, "pc: %h", sc.cpu.pc_out);
24.      $fdisplay(file_open, "instr: %h", sc.inst);
25.      $fdisplay(file_open, "regfile0: %h", sc.cpu.cpu_regfile.wyf_array_reg[0]);
26.      $fdisplay(file_open, "regfile1: %h", sc.cpu.cpu_regfile.wyf_array_reg[1]);
27.      $fdisplay(file_open, "regfile2: %h", sc.cpu.cpu_regfile.wyf_array_reg[2]);
28.      $fdisplay(file_open, "regfile3: %h", sc.cpu.cpu_regfile.wyf_array_reg[3]);
    end
  end
end

```

```

29.     $fdisplay(file_open, "regfile4: %h", sc.cpu.cpu_regfile.wyf_array_reg[4]);
30.     $fdisplay(file_open, "regfile5: %h", sc.cpu.cpu_regfile.wyf_array_reg[5]);
31.     $fdisplay(file_open, "regfile6: %h", sc.cpu.cpu_regfile.wyf_array_reg[6]);
32.     $fdisplay(file_open, "regfile7: %h", sc.cpu.cpu_regfile.wyf_array_reg[7]);
33.     $fdisplay(file_open, "regfile8: %h", sc.cpu.cpu_regfile.wyf_array_reg[8]);
34.     $fdisplay(file_open, "regfile9: %h", sc.cpu.cpu_regfile.wyf_array_reg[9]);
35.     $fdisplay(file_open, "regfile10: %h", sc.cpu.cpu_regfile.wyf_array_reg[10]);
36.     $fdisplay(file_open, "regfile11: %h", sc.cpu.cpu_regfile.wyf_array_reg[11]);
37.     $fdisplay(file_open, "regfile12: %h", sc.cpu.cpu_regfile.wyf_array_reg[12]);
38.     $fdisplay(file_open, "regfile13: %h", sc.cpu.cpu_regfile.wyf_array_reg[13]);
39.     $fdisplay(file_open, "regfile14: %h", sc.cpu.cpu_regfile.wyf_array_reg[14]);
40.     $fdisplay(file_open, "regfile15: %h", sc.cpu.cpu_regfile.wyf_array_reg[15]);
41.     $fdisplay(file_open, "regfile16: %h", sc.cpu.cpu_regfile.wyf_array_reg[16]);
42.     $fdisplay(file_open, "regfile17: %h", sc.cpu.cpu_regfile.wyf_array_reg[17]);
43.     $fdisplay(file_open, "regfile18: %h", sc.cpu.cpu_regfile.wyf_array_reg[18]);
44.     $fdisplay(file_open, "regfile19: %h", sc.cpu.cpu_regfile.wyf_array_reg[19]);
45.     $fdisplay(file_open, "regfile20: %h", sc.cpu.cpu_regfile.wyf_array_reg[20]);
46.     $fdisplay(file_open, "regfile21: %h", sc.cpu.cpu_regfile.wyf_array_reg[21]);
47.     $fdisplay(file_open, "regfile22: %h", sc.cpu.cpu_regfile.wyf_array_reg[22]);
48.     $fdisplay(file_open, "regfile23: %h", sc.cpu.cpu_regfile.wyf_array_reg[23]);
49.     $fdisplay(file_open, "regfile24: %h", sc.cpu.cpu_regfile.wyf_array_reg[24]);
50.     $fdisplay(file_open, "regfile25: %h", sc.cpu.cpu_regfile.wyf_array_reg[25]);
51.     $fdisplay(file_open, "regfile26: %h", sc.cpu.cpu_regfile.wyf_array_reg[26]);
52.     $fdisplay(file_open, "regfile27: %h", sc.cpu.cpu_regfile.wyf_array_reg[27]);
53.     $fdisplay(file_open, "regfile28: %h", sc.cpu.cpu_regfile.wyf_array_reg[28]);
54.     $fdisplay(file_open, "regfile29: %h", sc.cpu.cpu_regfile.wyf_array_reg[29]);
55.     $fdisplay(file_open, "regfile30: %h", sc.cpu.cpu_regfile.wyf_array_reg[30]);
56.     $fdisplay(file_open, "regfile31: %h", sc.cpu.cpu_regfile.wyf_array_reg[31]);
57. end
58. if (cnt > 500) begin
59.     $fclose(file_open);
60.     $stop;
61. end
62. end
63.
64. sccomp_dataflow sc(
65.     .clk_in(clk),
66.     .reset(rst),
67.     .inst(inst),
68.     .pc(pc)
69. );
70.
71. endmodule
72.

```

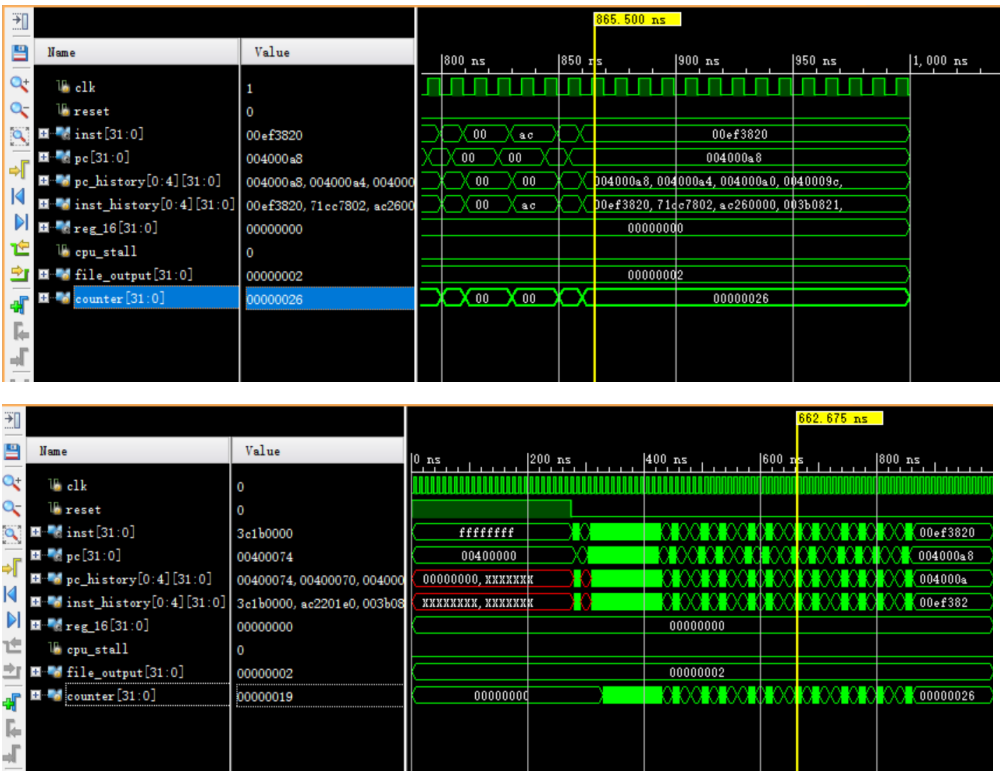
五、实验仿真的波形图及某时刻寄存器值的物理意义

1、动态流水线的波形图及某时刻寄存器值的物理意义

仿真波形图如下措施，从结果上看，可以正确实行流水线的断流，并且最终指令停留在最后的 00ef3820 处。

从下图可以清晰直观地看到在波形图中存在一些时刻，pc(程序计数器)和 inst(指令寄存器)处于稳定不变的状态。这种情况通常发生在流水线中的一些相关进行数据前推的情况下。这意味着流水线在这些时刻发生了暂停，以等待前面的指令完成执行并成功写回。一旦前一指令的相关和冲突被解决，流水线会继续执行，并在下一个时钟周期内读取并执行下一条指令。可以看到下图中 PC 值在一段时间里没有发生变化，在这段时间里，pc 和 instr 都保持不变，这是因为流水线正在处理数据相关的问题，暂停了一些周期。

这种流水线的断流和暂停机制是为了处理数据相关和指令冲突，确保指令的执行顺序正确，不会因为相关和冲突而导致错误的结果。通过这种方式，流水线能够在执行过程中动态地适应不同的指令流，并最终正确停留在目标地址，实现了高效而准确的指令执行。



2、寄存器的物理意义：

```

pc: 00400008
instr: 08100001
regfile0: 00000000
regfile1: 100100ec
regfile2: 00000000
regfile3: 00000001
regfile4: 00000000
regfile5: 000000f0
regfile6: 000006ea
regfile7: 000014bf
regfile8: 00000000
regfile9: 00000000
regfile10: 00000001
regfile11: 000000f0
regfile12: 0000003b
regfile13: 00000000
regfile14: 00000003
regfile15: 0087f096
regfile16: 9fd0b7ea
regfile17: 00000000
regfile18: 00000000
regfile19: 00000000
regfile20: 00000000
regfile21: 00000000
regfile22: 00000000
regfile23: 00000000
regfile24: 00000000
regfile25: 00000000
regfile26: 00000000
regfile27: 000000ec
regfile28: a0602880
regfile29: 00000000
regfile30: 00000000
regfile31: 00000000

```

在程序结束之后，可以看到 pc 停留在 00400008 处，regfile16 的值就是最终数组的计算结果储存了 D[59]的最终结果。C[59]+D[59]保存在 regfile28 中。

六、流水线 CPU 实验性能验证模型

```

int a[m],b[m],c[m],d[m];
a[0]=0;
b[0]=1;
a[i]=a[i-1]+i;
b[i]=b[i-1]+3i;
c[i]=
{
    a[i],          0≤i≤19
    a[i]+b[i], 20≤i≤39
    a[i]*b[i], 40≤i≤59
}
d[i]=
{
    b[i],          0≤i≤19
    a[i]*c[i], 20≤i≤39
    c[i]*b[i], 40≤i≤59
}

```

汇编语言如下：

```
1. .data
2. A:.space 240
3. B:.space 240
```

```
4. C:.space 240
5. D:.space 240
6. .text
7. sll $0,$0,0
8. exc: #exception
9. beq $30,$0,main
10. j exc
11.
12. main:
13. lui $30,0xffff    #enable exception
14. addi $2,$0,0      #a[i]
15. addi $3,$0,1      #b[i]
16. addi $15,$0,0     #c[i]
17. addi $16,$0,0     #d[i]
18. addi $5,$0,4      #counter
19. addi $6,$0,0      #a[i-1]
20. addi $7,$0,1      #b[i-1]
21. addi $10,$0,0     #flag for i<20 || i<40
22. addi $11,$0,240   #sum counts
23. addi $14,$0,3
24. #addi $30,$0,0
25.
26. # 把 0 1 0 0 ($2,...,$16) 分别存入 A B C D
27. lui $27,0x0000
28. addu $27,$27,$0
29. sw $2,A($27)
30. lui $27,0x0000
31. addu $27,$27,$0
32. sw $3,B($27)
33. lui $27,0x0000
34. addu $27,$27,$0
35. sw $2,C($27)
36. lui $27,0x0000
37. addu $27,$27,$0
38. sw $3,D($27)
39.
40. # 循环
41. loop:
42. ## $5(4) 除以 4 (=i) 存入 $12
43. srl $12,$5,2
```



```

44. # $6 加 i. 自此, $6 就是 a[i] 而不是 a[i-1] 了
45. add $6,$6,$12
46. # 把 a[i] 的内容存入 A[i] 中
47. lui $27,0x0000
48. addu $27,$27,$5
49. sw $6,A($27)
50. # $14 (3) 乘以 $5/4 (i) ( = 3i )
51. mul $15,$14,$12
52. # 把 $7 (b[i-1]) 的内容加上 3i, 存入 B[i]. 自此, $7 就是 b[i] 而不是 b[i-1]
53. add $7,$7,$15
54. lui $27,0x0000
55. addu $27,$27,$5
56. sw $7,B($27)
57. # $5 是否小于 80 (i 是否小于 20)? 记入 $10
58. slti $10,$5,80
59. # 若不是, 跳转
60. bne $10,1,c1
61.
62. # (0<=i<=19)
63. # 把 $6 的内容存入 C[i] 中 (c[i] = a[i])
64. lui $27,0x0000
65. addu $27,$27,$5
66. sw $6,C($27)
67. # 把 $7 的内容存入 D[i] 中 (d[i] = b[i])
68. lui $27,0x0000
69. addu $27,$27,$5
70. sw $7,D($27)
71. addi $15,$6,0 # $15 $16 分别赋值为 c[i] d[i]
72. addi $16,$7,0
73. j endc
74. c1: # (20<=i<=39)
75. # i 是否小于 40 ? 若不是, 跳转到 c2
76. slti $10,$5,160
77. addi $27,$0,1
78. bne $10,$27,c2
79. # C[i] = a[i] + b[i]
80. add $15,$6,$7
81. lui $27,0x0000
82. addu $27,$27,$5
83. sw $15,C($27)
84. # D[i] = a[i] * b[i]
85. mul $16,$15,$6
86. lui $27,0x0000

```

```

87. addu $27,$27,$5
88. sw $16,D($27)
89. j endc
90. c2: # (i>=40)
91. # C[i] = a[i] * b[i]
92. mul $15,$6,$7
93. lui $27,0x0000
94. addu $27,$27,$5
95. sw $15,C($27)
96. # D[i] = c[i] * b[i]
97. mul $16,$15,$7
98. lui $27,0x0000
99. addu $27,$27,$5
100. sw $16,D($27)
101.
102. endc:
103. addi $5,$5,4 # i = i + 1
104. bne $5,$11,loop # i = 60 不跳转
105. break

```

在通过 Mars 编译得到得到指令。

七、实验验算程序下板测试过程与实现

1、下板测试过程

在顶层模块加入七段数码管的，将 instruction 或者 regfile 中的部分值作为七段数码管的输入数据，进行显示。添加的七段数码管的文件如下：

```

1. `timescale 1ns / 1ps
2.
3. module seg7x16(
4.     input clk,
5.     input reset,
6.     input cs,
7.     input [31:0] i_data,
8.     output [7:0] o_seg,
9.     output [7:0] o_sel
10. );
11.
12.     reg [14:0] cnt;
13.     always @ (posedge clk, posedge reset)
14.         if (reset)

```

```

15.     cnt <= 0;
16.     else
17.         cnt <= cnt + 1'b1;
18.
19.     wire seg7_clk = cnt[14];
20.
21.     reg [2:0] seg7_addr;
22.
23.     always @ (posedge seg7_clk, posedge reset)
24.         if(reset)
25.             seg7_addr <= 0;
26.         else
27.             seg7_addr <= seg7_addr + 1'b1;
28.
29.     reg [7:0] o_sel_r;
30.
31.     always @ (*)
32.         case(seg7_addr)
33.             7 : o_sel_r = 8'b01111111;
34.             6 : o_sel_r = 8'b10111111;
35.             5 : o_sel_r = 8'b11011111;
36.             4 : o_sel_r = 8'b11101111;
37.             3 : o_sel_r = 8'b11110111;
38.             2 : o_sel_r = 8'b11111011;
39.             1 : o_sel_r = 8'b11111101;
40.             0 : o_sel_r = 8'b11111110;
41.         endcase
42.
43.     reg [31:0] i_data_store;
44.     always @ (posedge clk, posedge reset)
45.         if(reset)
46.             i_data_store <= 0;
47.         else if(cs)
48.             i_data_store <= i_data;
49.
50.     reg [7:0] seg_data_r;
51.     always @ (*)
52.         case(seg7_addr)
53.             0 : seg_data_r = i_data_store[3:0];
54.             1 : seg_data_r = i_data_store[7:4];
55.             2 : seg_data_r = i_data_store[11:8];
56.             3 : seg_data_r = i_data_store[15:12];
57.             4 : seg_data_r = i_data_store[19:16];
58.             5 : seg_data_r = i_data_store[23:20];

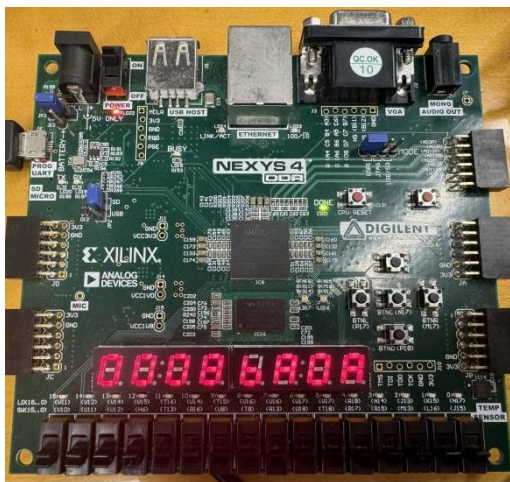
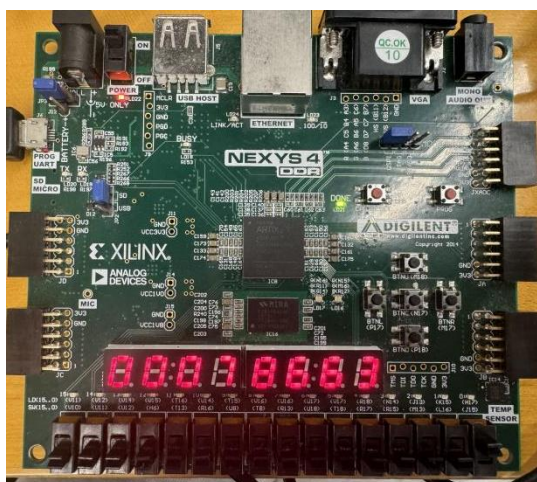
```

```

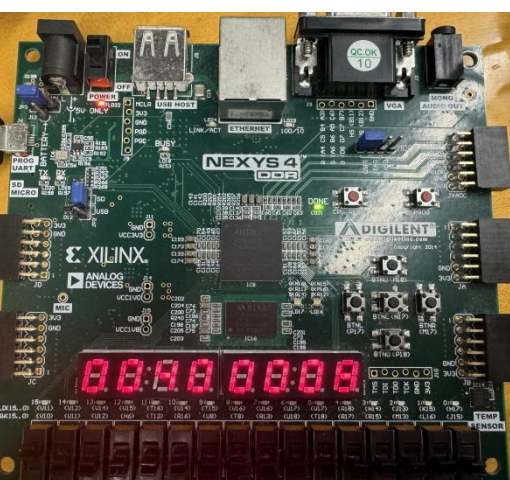
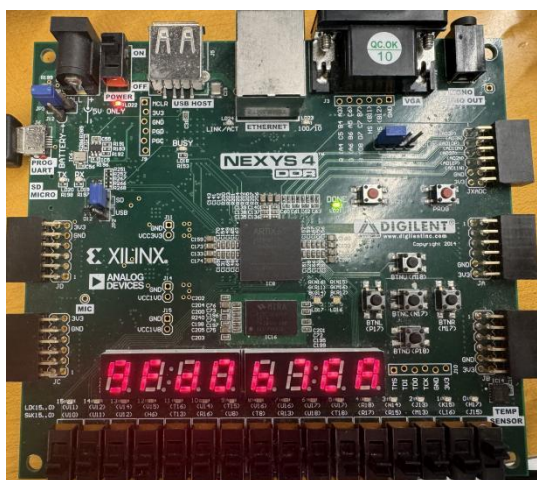
59.             6 : seg_data_r = i_data_store[27:24];
60.             7 : seg_data_r = i_data_store[31:28];
61.         endcase
62.
63.     reg [7:0] o_seg_r;
64.     always @ (posedge clk, posedge reset)
65.     if(reset)
66.         o_seg_r <= 8'hff;
67.     else
68.         case(seg_data_r)
69.             4'h0 : o_seg_r <= 8'hC0;
70.             4'h1 : o_seg_r <= 8'hF9;
71.             4'h2 : o_seg_r <= 8'hA4;
72.             4'h3 : o_seg_r <= 8'hB0;
73.             4'h4 : o_seg_r <= 8'h99;
74.             4'h5 : o_seg_r <= 8'h92;
75.             4'h6 : o_seg_r <= 8'h82;
76.             4'h7 : o_seg_r <= 8'hF8;
77.             4'h8 : o_seg_r <= 8'h80;
78.             4'h9 : o_seg_r <= 8'h90;
79.             4'hA : o_seg_r <= 8'h88;
80.             4'hB : o_seg_r <= 8'h83;
81.             4'hC : o_seg_r <= 8'hC6;
82.             4'hD : o_seg_r <= 8'hA1;
83.             4'hE : o_seg_r <= 8'h86;
84.             4'hF : o_seg_r <= 8'h8E;
85.         endcase
86.
87.     assign o_sel = o_sel_r;
88.     assign o_seg = o_seg_r;
89.
90. endmodule
91.

```

1、本 CPU 可以实现暂停，按钮 J15 置 1，可以实现流水线暂停。在暂停之后，按钮 J15 重新置 0，可以重新在断点进行计算。如下图所示，由按键开关产生一个暂停的中断，再次按键结束中断，继续运行后续的运算，并在数码管上动态显示运算值。



2、呈现最终的最终的计算结果 D[59]，按钮切换也可以显示当前的 PC 值。



八、流水线的性能指标定性分析

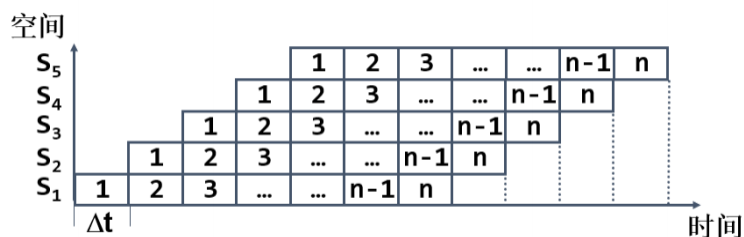
1、吞吐率：

流水线吞吐率的计算方式是通过单位时间内完成的指令数来表示。具体而言，吞吐率可以用以下公式表示：

$$\text{吞吐率} = \frac{\text{完成的指令数}}{\text{实行时间}}$$

其中，完成的指令数是指在一定时间内成功执行的指令数量，执行时间是从第一条指令进入流水线到最后一条指令完成执行的时间。吞吐率以每秒完成的指令数（Instructions Per Second，IPS）为单位。

当流水线分为 5 段组成时候，流水线理想情况的时空图如下：



而在本实验中，测试程序最终跑了 2375 条指令。从仿真结果可以看出，一个时钟周期是 6ns，，则由公式可算出，我编写的动态流水线的吞吐率为

$$\frac{2375 \text{ 条}}{17550\text{ns}} = \frac{0.135 \text{ 条}}{\text{ns}} = 135\text{MIPS}$$

2、加速比

流水线的加速比是指通过引入流水线设计相对于非流水线设计所实现的性能提升比率。加速比可以通过以下公式计算：

$$S = \frac{\text{执行时间(非流水线)}}{\text{执行时间(流水线)}} = \frac{T_0}{T_m}$$

如果在五阶段流水线中，假设一个阶段的执行时间为 Δt ，则流水线的加速比应该是：
$$S = \frac{5n \Delta t}{(5+n-1)\Delta t} = \frac{5n}{n+4}$$

故在我编写的动态流水线中，加速比应该进行如下计算：如果是非流水线设计，即一个指令需要 5 个周期来执行，则一共需要执行 $T_0 = 6\text{ns} \times 5 \times 2375 = 71250\text{ns}$ ；而流水线设计的 cpu 中，执行所有指令仅需 17550ns，计算得加速比为 4。

3、效率

理想情况下，五段流水线的效率为
$$E = \frac{5n \Delta t}{(5+n-1) \times 5\Delta t} = \frac{n}{n+4}$$

而在我编写的动态流水线中涉及到断流，效率应该进行如下计算：

$$E = \frac{5 \times 2375 \times 6\text{ns}}{5 \times 17550\text{ns}} = 0.881 \text{ 。}$$

4、相关与冲突分析

4.1 本动态流水线涉及到两种冲突

1、数据相关：

临近的两条指令数据相关，或者间隔一条指令的两条指令之间有数据相关的情况。就是说前一条指令的目的操作数是下一条或者下两条的指令的源操作数。具体表现为 ID 和 EX 段的冲突与 ID 和 MEM 段的冲突。对于上面的这些情况，我们采用数据前推的方法，在 IF 时，检测相应的数据段进行是否有上面的这些情况存在，如果存在这种情况，我们将 `data_forward_rt`，`data_forward_rs` 重新赋值，作为 `regfile` 的两个地址选择，前推的数据送入执行段，直接参与后面指令的运算。

2、跳转指令冲突：

冲突类型：在流水线中，如果当前处于 IF 段的跳转指令，需要判断上一条指令（即当前在 ID、EX、MEM 段的指令）是否修改了跳转指令相关的寄存器值。

冲突处理：如果有相关的寄存器写入操作，可以暂停跳转指令的执行。

4.2 代码中的具体实现方式包括：

使用信号（如 `Reg_ID_W`、`Reg_EX_W`、`Reg_MEM_W`）来表示当前流水线阶段是否有可能写入寄存器，以及写入的寄存器地址（如 `Reg_ID_W_address`、`Reg_EX_W_address`、`Reg_MEM_W_address`）。

使用 `JUMP_halt` 信号来表示跳转指令冲突，以及 `RAW_halt` 信号来表示读写冲突。同时，通过逻辑判断来检测和处理冲突情况，进而暂停流水线的执行，确保数据的正确性，

5、CPU 运行时间

由本章中前几个小结的分析可以得出，本实验的 `cpu` 运行时间为 618ns。各种数据相关和指令冲突会导致流水线暂停，`cpu` 运行时间变长。

不失一般性，CPU 的运行时间与以下几个方面相关：

5.1、流水线深度：流水线深度是指流水线中有多少个阶段。流水线的深度越大，理论上并行执行的指令越多，但同时也可能增加由于流水线暂停引起的延迟。

5.2、指令集：不同类型的指令可能有不同的执行时长，而且一些指令可能引起流水线暂停。通过分析指令集的性质，可以更好地理解指令的执行时间以及可能的延迟。

5.3、数据相关性：通过分析数据相关性，可以考虑采用一些技术来减少数据相关性，比如乱序执行、超标量架构等。

5.4、分支预测准确性：分支指令的准确预测对于流水线的性能至关重要。高效的分支预测机制可以减少由于分支带来的流水线暂停。

5.5、硬件性能：CPU 的硬件性能，包括时钟频率、缓存大小等，对运行时长有直接影响。更高的时钟频率和更大的缓存通常能提高处理速度。

5.6、程序特性：考虑运行的实际程序，特别是是否有循环、分支等结构，这些结构可能会导致流水线暂停，影响整体的运行时长。

6. 存储器空间的使用

```
pc: 00400008
instr: 08100001
regfile0: 00000000
regfile1: 100100ec
regfile2: 00000000
regfile3: 00000001
regfile4: 00000000
regfile5: 000000f0
regfile6: 000006ea
regfile7: 000014bf
regfile8: 00000000
regfile9: 00000000
regfile10: 00000001
regfile11: 000000f0
regfile12: 0000003b
regfile13: 00000000
regfile14: 00000003
regfile15: 0087f096
regfile16: 9fd0b7ea
regfile17: 00000000
regfile18: 00000000
regfile19: 00000000
regfile20: 00000000
regfile21: 00000000
regfile22: 00000000
regfile23: 00000000
regfile24: 00000000
regfile25: 00000000
regfile26: 00000000
regfile27: 000000ec
regfile28: a0602880
regfile29: 00000000
regfile30: 00000000
regfile31: 00000000
```

Regfile16 中储存了 D[59]的最终结果，其他寄存器存储中间变量。

八、总结与体会

通过编写 31 条指令的动态流水线 CPU，我深刻领悟了多指令并行执行核心理念。将指令执行过程划分为五个独立的阶段，即指令提取、译码、执行、访存和写回，显著提升了整体系统的效率与性能。

在流水线 CPU 的设计中，冲突处理显现出至关重要的地位。我不仅需考虑由数据相关引发的冲突，还需处理跳转指令引起的控制冲突。在这个项目中，我采用了流水线暂停的策略来解决冲突，通过合理的设计和处理，确保了指令的顺利执行。冲突的妥善处理不仅维护了程序的正确性，还确保了整个系统的稳健性。

综合而言，通过对系统结构的优化，我深刻认识到系统结构设计对整个计算机体系性能提升的至关重要性。巧妙设计的流水线结构极大地提高了工作效率，生动展示了系统结构设计在计算机科学领域的关键作用。这个项目让我更加深入地理解了流水线 CPU 的工作原理，并对计算机体系结构有了更为全面深刻的认知。

九、附件（所有程序）

1、sccomp_dataflow

```
1. module sccomp_dataflow(  
2.     input clk_in,  
3.     input reset,  
4.     output [31:0] inst,  
5.     output [31:0] pc  
6.  
7. );  
8.  
9. wire [31:0] IMEM_Address;  
10.  
11. wire [31:0] instruction;  
12. assign inst = instruction;  
13.  
14. IMEM IMEM(  
15.     .addr(IMEM_Address[10:0]),  
16.     .output_data(instruction)  
17. );  
18.  
19.
```

```

20. CPU cpu(
21.     .clk(clk_in),
22.     .rst(reset),
23.     .IMEM_instruction(instruction),
24.     .pc_out(pc)
25. );
26.
27. assign IMEM_Address = (pc - 32'h0040_0000)/4;
28.
29. //seg7x16 seven(
30. //     .clk(clk_in),
31. //     .reset(reset),
32. //     .cs(1'b1),
33. //     .i_data(4),
34. //     .o_seg(oSeg),
35. //     .o_sel(oSel)
36. //);
37.
38. endmodule
39.

```

2、CPU

```

1. `timescale 1ns / 1ps
2. `include "def.vh"
3. module CPU(
4.     input clk,
5.     input rst,
6.     input [31:0] IMEM_instruction,
7.     output [31:0] pc_out
8.
9. );
10. reg JUMP_halt;
11. reg RAW_halt;
12.
13. wire [31:0] IFInstruction;
14. wire [31:0] if_ir = IFInstruction;
15.
16. wire [31:0] ifID_ir;
17. wire [31:0] ID_ir = ifID_ir;
18.
19. wire [31:0] IDEX_ir;
20. wire [31:0] EX_ir = IDEX_ir;
21.

```

```

22. wire [31:0]EX_MEM_ir;
23. wire [31:0]MEM_ir = EX_MEM_ir;
24.
25. wire [31:0]MEM_WBack_ir;
26. wire [31:0]WBack_ir = MEM_WBack_ir;
27.
28. // 首先进行指令译码
29. // IF
30. wire [5:0]if_op ;
31. wire [4:0]if_shamt;
32. wire [5:0]if_func ;
33. wire [15:0]if_imm16 ;
34. wire [4:0]if_Rsc ;
35. wire [4:0]if_Rtc;
36. wire [4:0]if_Rdc ;
37.
38.
39. wire if_addu, if_sub, if_subu;
40. wire if_and, if_or, if_xor, if_nor, if_slt, if_sltu, if_sll, if_srl, if_sra,
if_sllv, if_srlv, if_srav, if_jr;
41.
42. // Immediate instructions
43. wire if_addi, if_addiu, if_andi, if_ori, if_xori, if_lw, if_sw, if_beq, if_bne,
if_slti, if_sltiu, if_lui;
44.
45. // Jump instructions
46. wire if_j, if_jal;
47.
48. IF_Decoder IF_Decoder (
49.   .if_ir(if_ir),
50.   .if_op(if_op),
51.   .if_shamt(if_shamt),
52.   .if_func(if_func),
53.   .if_imm16(if_imm16),
54.   .if_Rsc(if_Rsc),
55.   .if_Rtc(if_Rtc),
56.   .if_Rdc(if_Rdc),
57.   .if_add(if_add),
58.   .if_addu(if_addu),
59.   .if_sub(if_sub),
60.   .if_subu(if_subu),
61.   .if_and(if_and),
62.   .if_or(if_or),
63.   .if_xor(if_xor),

```

```

64. .if_nor(if_nor),
65. .if_slt(if_slt),
66. .if_sltu(if_sltu),
67. .if_sll(if_sll),
68. .if_srl(if_srl),
69. .if_sra(if_sra),
70. .if_sllv(if_sllv),
71. .if_srlv(if_srlv),
72. .if_srav(if_srav),
73. .if_jr(if_jr),
74. .if_addi(if_addi),
75. .if_addiu(if_addiu),
76. .if_andi(if_andi),
77. .if_ori(if_ori),
78. .if_xori(if_xori),
79. .if_lw(if_lw),
80. .if_sw(if_sw),
81. .if_beq(if_beq),
82. .if_bne(if_bne),
83. .if_slti(if_slti),
84. .if_sltiu(if_sltiu),
85. .if_lui(if_lui),
86. .if_j(if_j),
87. .if_jal(if_jal)
88. );
89.
90. // ID 段
91. wire [5:0] ID_op = ID_ir[31:26];
92. wire [4:0] ID_shamt = ID_ir[10:6];
93. wire [5:0] ID_func = ID_ir[5:0];
94. wire [15:0] ID_imm16 = ID_ir[15:0];
95. wire [4:0] ID_Rsc = ID_ir[25:21];
96. wire [4:0] ID_Rtc = ID_ir[20:16];
97. wire [4:0] ID_Rdc = ID_ir[15:11];
98.
99. `define ID_OP_COMMON(op) (ID_op == `OP_COMMON && ID_func == op)
100. `define ID_OP(op) (ID_op == op)
101.
102. // ID stage ALU instructions
103. wire ID_add = `ID_OP_COMMON(`FUNCT_ADD);
104. wire ID_addu = `ID_OP_COMMON(`FUNCT_ADDU);
105. wire ID_sub = `ID_OP_COMMON(`FUNCT_SUB);
106. wire ID_subu = `ID_OP_COMMON(`FUNCT_SUBU);
107. wire ID_and = `ID_OP_COMMON(`FUNCT_AND);

```

```

108. wire ID_or = `ID_OP_COMMON(`FUNCT_OR);
109. wire ID_xor = `ID_OP_COMMON(`FUNCT_XOR);
110. wire ID_nor = `ID_OP_COMMON(`FUNCT_NOR);
111. wire ID_slt = `ID_OP_COMMON(`FUNCT_SLT);
112. wire ID_sltu = `ID_OP_COMMON(`FUNCT_SLTU);
113. wire ID_sll = `ID_OP_COMMON(`FUNCT_SLL);
114. wire ID_srl = `ID_OP_COMMON(`FUNCT_SRL);
115. wire ID_sra = `ID_OP_COMMON(`FUNCT_SRA);
116. wire ID_sllv = `ID_OP_COMMON(`FUNCT_SLLV);
117. wire ID_srlv = `ID_OP_COMMON(`FUNCT_SRLV);
118. wire ID_srav = `ID_OP_COMMON(`FUNCT_SRAV);
119. wire ID_jr = `ID_OP_COMMON(`FUNCT_JR);
120.
121. // ID stage Immediate instructions
122. wire ID_addi = `ID_OP(`OP_ADDI);
123. wire ID_addiu = `ID_OP(`OP_ADDIU);
124. wire ID_andi = `ID_OP(`OP_ANDI);
125. wire ID_ori = `ID_OP(`OP_ORI);
126. wire ID_xori = `ID_OP(`OP_XORI);
127. wire ID_lw = `ID_OP(`OP_LW);
128. wire ID_sw = `ID_OP(`OP_SW);
129. wire ID_beq = `ID_OP(`OP_BEQ);
130. wire ID_bne = `ID_OP(`OP_BNE);
131. wire ID_slti = `ID_OP(`OP_SLTI);
132. wire ID_sltiu = `ID_OP(`OP_SLTIU);
133. wire ID_lui = `ID_OP(`OP_LUI);
134.
135. // ID stage Jump instructions
136. wire ID_j = `ID_OP(`OP_J);
137. wire ID_jal = `ID_OP(`OP_JAL);
138.
139.
140. // EX 段
141. wire [5:0] EX_op = EX_ir[31:26];
142. wire [4:0] EX_shamt = EX_ir[10:6];
143. wire [5:0] EX_func = EX_ir[5:0];
144. wire [15:0] EX_imm16 = EX_ir[15:0];
145. wire [4:0] EX_Rsc = EX_ir[25:21];
146. wire [4:0] EX_Rtc = EX_ir[20:16];
147. wire [4:0] EX_Rdc = EX_ir[15:11];
148.
149. `define EX_OP_COMMON(func) (EX_op == 6'b000000 && EX_func == func)
150. `define EX_OP(op) (EX_op == op)
151.

```

```

152. // EX stage ALU instructions
153. wire EX_add = `EX_OP_COMMON(`FUNCT_ADD);
154. wire EX_addu = `EX_OP_COMMON(`FUNCT_ADDU);
155. wire EX_sub = `EX_OP_COMMON(`FUNCT_SUB);
156. wire EX_subu = `EX_OP_COMMON(`FUNCT_SUBU);
157. wire EX_and = `EX_OP_COMMON(`FUNCT_AND);
158. wire EX_or = `EX_OP_COMMON(`FUNCT_OR);
159. wire EX_xor = `EX_OP_COMMON(`FUNCT_XOR);
160. wire EX_nor = `EX_OP_COMMON(`FUNCT_NOR);
161. wire EX_slt = `EX_OP_COMMON(`FUNCT_SLT);
162. wire EX_sltu = `EX_OP_COMMON(`FUNCT_SLTU);
163. wire EX_sll = `EX_OP_COMMON(`FUNCT_SLL);
164. wire EX_srl = `EX_OP_COMMON(`FUNCT_SRL);
165. wire EX_sra = `EX_OP_COMMON(`FUNCT_SRA);
166. wire EX_sllv = `EX_OP_COMMON(`FUNCT_SLLV);
167. wire EX_srlv = `EX_OP_COMMON(`FUNCT_SRLV);
168. wire EX_srav = `EX_OP_COMMON(`FUNCT_SRAV);
169. wire EX_jr = `EX_OP_COMMON(`FUNCT_JR);
170.
171. // EX stage Immediate instructions
172. wire EX_addi = `EX_OP(`OP_ADDI);
173. wire EX_addiu = `EX_OP(`OP_ADDIU);
174. wire EX_andi = `EX_OP(`OP_ANDI);
175. wire EX_ori = `EX_OP(`OP_ORI);
176. wire EX_xori = `EX_OP(`OP_XORI);
177. wire EX_lw = `EX_OP(`OP_LW);
178. wire EX_sw = `EX_OP(`OP_SW);
179. wire EX_beq = `EX_OP(`OP_BEQ);
180. wire EX_bne = `EX_OP(`OP_BNE);
181. wire EX_slti = `EX_OP(`OP_SLTI);
182. wire EX_sltiu = `EX_OP(`OP_SLTIU);
183. wire EX_lui = `EX_OP(`OP_LUI);
184.
185. // EX stage Jump instructions
186. wire EX_j = `EX_OP(`OP_J);
187. wire EX_jal = `EX_OP(`OP_JAL);
188.
189.
190. // MEM 段
191. wire [5:0] MEM_op = MEM_ir[31:26];
192. wire [4:0] MEM_shamt = MEM_ir[10:6];
193. wire [5:0] MEM_func = MEM_ir[5:0];
194. wire [15:0] MEM_imm16 = MEM_ir[15:0];
195. wire [4:0] MEM_Rsc = MEM_ir[25:21];

```

```

196. wire [4:0] MEM_Rtc = MEM_ir[20:16];
197. wire [4:0] MEM_Rdc = MEM_ir[15:11];
198.
199. `define MEM_OP_COMMON(func) (MEM_op == 6'b000000 && MEM_func == func)
200. `define MEM_OP(op) (MEM_op == op)
201.
202. // ME stage ALU instructions
203. wire MEM_add = `MEM_OP_COMMON(`FUNCT_ADD);
204. wire MEM_addu = `MEM_OP_COMMON(`FUNCT_ADDU);
205. wire MEM_sub = `MEM_OP_COMMON(`FUNCT_SUB);
206. wire MEM_subu = `MEM_OP_COMMON(`FUNCT_SUBU);
207. wire MEM_and = `MEM_OP_COMMON(`FUNCT_AND);
208. wire MEM_or = `MEM_OP_COMMON(`FUNCT_OR);
209. wire MEM_xor = `MEM_OP_COMMON(`FUNCT_XOR);
210. wire MEM_nor = `MEM_OP_COMMON(`FUNCT_NOR);
211. wire MEM_slt = `MEM_OP_COMMON(`FUNCT_SLT);
212. wire MEM_sltu = `MEM_OP_COMMON(`FUNCT_SLTU);
213. wire MEM_sll = `MEM_OP_COMMON(`FUNCT_SLL);
214. wire MEM_srl = `MEM_OP_COMMON(`FUNCT_SRL);
215. wire MEM_sra = `MEM_OP_COMMON(`FUNCT_SRA);
216. wire MEM_sllv = `MEM_OP_COMMON(`FUNCT_SLLV);
217. wire MEM_srlv = `MEM_OP_COMMON(`FUNCT_SRLV);
218. wire MEM_srav = `MEM_OP_COMMON(`FUNCT_SRAV);
219. wire MEM_jr = `MEM_OP_COMMON(`FUNCT_JR);
220.
221. // ME stage Immediate instructions
222. wire MEM_addi = `MEM_OP(`OP_ADDI);
223. wire MEM_addiu = `MEM_OP(`OP_ADDIU);
224. wire MEM_andi = `MEM_OP(`OP_ANDI);
225. wire MEM_ori = `MEM_OP(`OP_ORI);
226. wire MEM_xori = `MEM_OP(`OP_XORI);
227. wire MEM_lw = `MEM_OP(`OP_LW);
228. wire MEM_sw = `MEM_OP(`OP_SW);
229. wire MEM_beq = `MEM_OP(`OP_BEQ);
230. wire MEM_bne = `MEM_OP(`OP_BNE);
231. wire MEM_slti = `MEM_OP(`OP_SLTI);
232. wire MEM_sltiu = `MEM_OP(`OP_SLTIU);
233. wire MEM_lui = `MEM_OP(`OP_LUI);
234.
235. // ME stage Jump instructions
236. wire MEM_j = `MEM_OP(`OP_J);
237. wire MEM_jal = `MEM_OP(`OP_JAL);
238.
239.

```

```

240. // WBack 段
241. wire [5:0] WBack_op = WBack_ir[31:26];
242. wire [4:0] WBack_shamt = WBack_ir[10:6];
243. wire [5:0] WBack_func = WBack_ir[5:0];
244. wire [15:0] WBack_imm16 = WBack_ir[15:0];
245. wire [4:0] WBack_Rsc = WBack_ir[25:21];
246. wire [4:0] WBack_Rtc = WBack_ir[20:16];
247. wire [4:0] WBack_Rdc = WBack_ir[15:11];
248.
249. `define WBack_OP_COMMON(func) (WBack_op == 6'b000000 && WBack_func == func)
250. `define WBack_OP(op) (WBack_op == op)
251.
252. // WBack stage ALU instructions
253. wire WBack_add = `WBack_OP_COMMON(`FUNCT_ADD);
254. wire WBack_addu = `WBack_OP_COMMON(`FUNCT_ADDU);
255. wire WBack_sub = `WBack_OP_COMMON(`FUNCT_SUB);
256. wire WBack_subu = `WBack_OP_COMMON(`FUNCT_SUBU);
257. wire WBack_and = `WBack_OP_COMMON(`FUNCT_AND);
258. wire WBack_or = `WBack_OP_COMMON(`FUNCT_OR);
259. wire WBack_xor = `WBack_OP_COMMON(`FUNCT_XOR);
260. wire WBack_nor = `WBack_OP_COMMON(`FUNCT_NOR);
261. wire WBack_slt = `WBack_OP_COMMON(`FUNCT_SLT);
262. wire WBack_sltu = `WBack_OP_COMMON(`FUNCT_SLTU);
263. wire WBack_sll = `WBack_OP_COMMON(`FUNCT_SLL);
264. wire WBack_srl = `WBack_OP_COMMON(`FUNCT_SRL);
265. wire WBack_sra = `WBack_OP_COMMON(`FUNCT_SRA);
266. wire WBack_sllv = `WBack_OP_COMMON(`FUNCT_SLLV);
267. wire WBack_srlv = `WBack_OP_COMMON(`FUNCT_SRLV);
268. wire WBack_srav = `WBack_OP_COMMON(`FUNCT_SRAV);
269. wire WBack_jr = `WBack_OP_COMMON(`FUNCT_JR);
270.
271. // 18-29 WBack_op2-4 bits
272. wire WBack_addi = `WBack_OP(6'b001000);
273. wire WBack_addiu = `WBack_OP(6'b001001);
274. wire WBack_andi = `WBack_OP(6'b001100);
275. wire WBack_ori = `WBack_OP(6'b001101);
276. wire WBack_xori = `WBack_OP(6'b001110);
277. wire WBack_lw = `WBack_OP(6'b100011);
278. wire WBack_sw = `WBack_OP(6'b101011);
279. wire WBack_beq = `WBack_OP(6'b000100);
280. wire WBack_bne = `WBack_OP(6'b000101);
281. wire WBack_slti = `WBack_OP(6'b001010);
282. wire WBack_sltiu = `WBack_OP(6'b001011);
283. wire WBack_lui = `WBack_OP(6'b001111);

```



```

284.
285. // 30-31 WBack_op 2 bits
286. wire WBack_j = `WBack_OP(6'b000010);
287. wire WBack_jal = `WBack_OP(6'b000011);
288.
289.
290. // IF
291. // pc
292. wire [31:0] If_Pc_MUX_Out;
293. wire [31:0] IF_Pc_Out_Final;
294. assign pc_out = IF_Pc_Out_Final;
295. assign IFInstruction = IMEM_instruction;
296. //wire pc_ena = 1'b1;
297.
298. pc_reg pc_reg(
299.     .clk(clk),
300.     .rst(rst),
301.     .ena(1'b1),
302.     .in_data(If_Pc_MUX_Out),
303.     .out_data(IF_Pc_Out_Final)
304. );
305.
306. //ID_jr -> ID_regfile_d1
307. //ID_bne ID_beq -> IF_ID_NPC+ID_EXT18,
308. //ID_jal||ID_j -> {IF_ID_NPC[31:28],ifID_ir[25:0],2'b0}
309. //其他 IF_ID_NPC
310.
311. //mux_PC
312. // IF/ID 段信号，连接段都是寄存器
313. wire [31:0]IF_ID_NPC;
314. wire [31:0] ID_regfile_d1; // RegFile 第一个数据 (Rs)
315. wire [31:0] ID_regfile_d2; // RegFile 第二个数据 (Rt)
316. assign If_Pc_MUX_Out= ID_jr ? ID_regfile_d1 : (((ID_bne & !ID_EQUAL)|| (ID_beq &
ID_EQUAL)) ? IF_ID_NPC+ID_EXT18 : (( (ID_jal||ID_j) ?
{IF_ID_NPC[31:28],ifID_ir[25:0],2'b0} : IF_ID_NPC)));
317.
318. // 段与段之间的信号连接，reg 连接赋值
319. // IF/ID 段
320.
321. IF_ID if_ID_module (
322.     .clk(clk),
323.     .rst(rst),
324.     .JUMP_halt(JUMP_halt),
325.     .RAW_halt(RAW_halt),

```

```

326. .If_instruction(IFInstruction),
327. .If_PC(IF_Pc_Out_Final),
328. .IfId_Npc(IF_ID_NPC), // Connect to your existing signal
329. .IfId_Ir(ifID_ir) // Connect to your existing signal
330. );
331.
332. // WBack 段信号
333. wire [31:0]WBack_mux_data_out;
334. wire [4:0]WBack_mux_addr_out;
335. wire WBack_rf_wena;
336.
337.
338. // regfile
339. assign ena=1'b1;
340. cpu_regfile cpu_regfile(
341. .reg_clk(clk), // 时钟
342. .reg_rst(rst), // 复位
343. .reg_ena(ena), // 使能信号
344. .reg_wena(WBack_rf_wena), // 写使能信号
345. .raddr1(ID_Rsc), // 读地址 1
346. .raddr2(ID_Rtc), // 读地址 2
347. .waddr(WBack_mux_addr_out), // 写地址
348. .wdata(WBack_mux_data_out), // 写数据
349. .rdata1(ID_regfile_d1), // 读数据 1
350. .rdata2(ID_regfile_d2) // 读数据 2
351. );
352.
353.
354. wire ID_EQUAL;
355. assign ID_EQUAL = (ID_regfile_d1 == ID_regfile_d2);
356.
357. //alua alub
358. wire [2:0] ID_mux_my_ALUa_selector;
359. assign ID_mux_my_ALUa_selector[0]=ID_sra||ID_sr1||ID_s11; //data0 ->ID_EXT5
360. assign ID_mux_my_ALUa_selector[1]=ID_jal; //data1 -> IF_ID_NPC
361. assign ID_mux_my_ALUa_selector[2]=!(ID_sra||ID_sr1||ID_s11||ID_jal); // data2
->ID_regfile_d1
362.
363. wire [31:0] ID_EXT5;
364. wire [31:0] ID_EXT16;
365. wire [31:0] ID_EXT18;
366. wire [31:0]EXT16;
367. wire [31:0] EXT16_sign;
368.

```

```

369. assign EXT16_sign = {{16{ifID_ir[15]}},ifID_ir[15:0]}; //有符号扩展
370. assign EXT16 = { {16{1'b0}}, ifID_ir[15:0]};
371. assign ID_EXT16= (ID_addi||ID_addiu||ID_slti||ID_sltiu||ID_lw) ? EXT16_sign :
EXT16;
372.
373. wire [31:0] EXT5;
374. wire [31:0] EXT18;
375. assign ID_EXT5 = { 27'b0,ifID_ir[10:6]};
376.
377. assign ID_EXT18 = {{14{ifID_ir[15]}},{ifID_ir[15:0],2'b00}};
378.
379.
380.
381. wire [31:0] ID_mux_my_ALUa_out;
382. wire [31:0] ID_mux_my_ALUb_out;
383. ThreeInputSelector MUX_ALUA(ID_EXT5,IF_ID_NPC,ID_regfile_d1,
384.                               ID_mux_my_ALUa_selector,
385.                               ID_mux_my_ALUa_out);
386.
387. //
388. wire [2:0] ID_mux_my_ALUb_selector;
389. assign ID_mux_my_ALUb_selector[0] = ID_jal; //data0 -> 4
390. assign ID_mux_my_ALUb_selector[1] =
ID_addi||ID_addiu||ID_andi||ID_ori||ID_sltiu||ID_lui||ID_xori||ID_slti||ID_lw||ID_sw;
//data1 ->ID_EXT16
391. assign ID_mux_my_ALUb_selector[2]
= !(ID_addi||ID_addiu||ID_andi||ID_ori||ID_sltiu||ID_lui||ID_xori||ID_slti||ID_lw||ID_sw
||ID_jal); //data2 -> ID_regfile_d2
392.
393. ThreeInputSelector MUX_my_ALUB(4,ID_EXT16,ID_regfile_d2,
394.                               ID_mux_my_ALUb_selector,
395.                               ID_mux_my_ALUb_out);
396.
397. assign ID_pass_data = (ID_sw)?ID_regfile_d2:32'bz;
398.
399. // ID_EX
400. wire [31:0]IDEX_my_ALUa;
401. wire [31:0]IDEX_my_ALUb;
402.
403. wire [31:0] ID_EX_pass_data;
404.
405. ID_EX ID_ex_instance(
406.     .clk(clk),
407.     .rst(rst),

```

```

408.     .RAW_halt(RAW_halt),
409.     .id_mux_alua_out(ID_mux_my_ALUa_out),
410.     .id_mux_alub_out(ID_mux_my_ALUb_out),
411.     .id_pass_data(ID_pass_data),
412.     .ifid_ir(ifID_ir),
413.     .IdExAluA(IDEX_my_ALUa),
414.     .IdExAluB(IDEX_my_ALUb),
415.     .IdEx_transfer(ID_EX_pass_data),
416.     .IdEx_Ir(IDEX_ir)
417. );
418.
419. // EX
420. wire negative,overflow,carry,zero;
421.
422. wire [31:0]EX_my_ALU_out; //加法器
423. wire [3:0] my_ALUC;
424.
425. // Use conditional assignments to set individual bits
426. assign my_ALUC[3] = (EX_slt || EX_sltu || EX_sllv || EX_srlv || EX_srav || EX_sll
|| EX_srl || EX_sra || EX_slti || EX_sltiu || EX_lui) ? 1'b1 : 1'b0;
427. assign my_ALUC[2] = (EX_and || EX_or || EX_xor || EX_nor || EX_sllv || EX_srlv ||
EX_srav || EX_sll || EX_srl || EX_sra || EX_andi || EX_ori || EX_xori) ? 1'b1 : 1'b0;
428. assign my_ALUC[1] = (EX_add || EX_sub || EX_xor || EX_nor || EX_slt || EX_sltu ||
EX_sllv || EX_addi || EX_xori || EX_slti || EX_sltiu || EX_beq || EX_bne) ? 1'b1 : 1'b0;
429. assign my_ALUC[0] = (EX_sub || EX_subu || EX_or || EX_nor || EX_slt || EX_srlv ||
EX_srl || EX_ori || EX_slti || EX_beq || EX_bne) ? 1'b1 : 1'b0;
430.
431.
432. my_ALU ALU_instance(
433.     .operator_a(IDEX_my_ALUa),
434.     .operator_b(IDEX_my_ALUb),
435.     .alu_controller(my_ALUC),
436.     .result(EX_my_ALU_out),
437.     .zero(zero),
438.     .carry(carry),
439.     .negative(negative),
440.     .overflow(overflow)
441. );
442.
443.
444. // EX_MEM
445.
446. wire EX_MEM_overflowFlag;
447. wire [31:0] EX_MEM_pass_data;

```

```

448.
449. wire [31:0]EX_MEM_my_ALU_out;
450. wire [31:0]EX_MEM_my_ALUb;
451.
452. EX_MEM EX_MEM_inst (
453.     .clk(clk),
454.     .rst(rst),
455.     .ExAluOut(EX_my_ALU_out),          // Connect to the appropriate signal
456.     .IdExPassData(ID_EX_pass_data),
457.     .IdExIr(IDEX_ir),
458.     .overflow(overflow),
459.     .IdExAlub(IDEX_my_ALUb),
460.
461.     .ExMeAluOut(EX_MEM_my_ALU_out),
462.     .ExMePassData(EX_MEM_pass_data),
463.     .ExMeIr(EX_MEM_ir),
464.     .ExMeOverflowFlag(EX_MEM_overflowFlag),
465.     .ExMeAlub(EX_MEM_my_ALUb)
466. );
467.
468.
469. //MEM
470. assign DM_W = MEM_sw;
471. assign DM_R = MEM_lw;
472. assign DM_wdata = EX_MEM_pass_data;
473. assign DM_addr = EX_MEM_my_ALU_out;
474.
475. //MEM_WB
476. // MEM 段信号
477. // WBack 段信号
478.
479. wire [31:0]MEM_dmem_out;
480. wire [31:0]MEM_dm_wdata;
481. wire [31:0]MEM_dm_addr;
482.
483. wire [31:0]MEM_WBack_my_ALU_out;
484.
485. wire [31:0]MEM_WBack_my_ALUb;
486. reg [31:0]MEM_WBack_mdr;
487.
488. wire MEM_WBack_overflowFlag;
489. wire [31:0] MEM_WBack_pass_data;
490.
491. MEM_WB mem_wb_inst (

```

```

492. .clk(clk),
493. .rst(rst),
494. .MeLw(MEM_lw),
495. .MeDmemOut(MEM_dmem_out),
496. .ExMeAluOut(EX_MEM_my_ALU_out),
497. .ExMePassData(EX_MEM_pass_data),
498. .ExMeIr(EX_MEM_ir),
499. .ExMeOverflowFlag(EX_MEM_overflowFlag),
500. .ExMeAlub(EX_MEM_my_ALUb),
501. .MemWbAluOut(MEM_WBack_my_ALU_out),
502. .MeWbPassData(MEM_WBack_pass_data),
503. .MemWbIr(MEM_WBack_ir),
504. .MemWbOverflowFlag(MEM_WBack_overflowFlag),
505. .MemWbAlub( MEM_WBack_my_ALUb)
506. );
507.
508.
509. //WBack
510. assign WBack_rf_wena =
((!WBack_sw)&&(!WBack_beq)&&(!WBack_bne)&&(!WBack_j)&&(!WBack_jr)) ? 1'b1 : 1'b0; //向
regfile 写使能信号
511.
512. assign WBack_mux_data_out = WBack_lw ? MEM_WBack_mdr : MEM_WBack_my_ALU_out;
513.
514. assign M1 =
(WBack_addi||WBack_addiu||WBack_andi||WBack_ori||WBack_sltiu||WBack_lui||WBack_xori||WBa
ck_lw||WBack_slti) ? 2'b00 : ( (WBack_jal) ? 2'b10 : 2'b01);
515. assign WBack_mux_addr_out = (M1 == 2'b00) ? MEM_WBack_ir[20:16] : ((M1 == 2'b01) ?
MEM_WBack_ir[15:11] : 5'd31);
516.
517.
518. //判断写后读的数据冲突 前一指令还没写入（当前 MEM 段）下一指令有可能读（当前 ID 段）了
519. //计算 MEM 段中有可能写入的指令
520. wire Reg_ID_W ;
521. assign Reg_ID_W = ID_addi || ID_addiu || ID_andi || ID_ori || ID_sltiu || ID_lui ||
ID_xori || ID_slti || ID_sltiu || ID_lui ||
522. ID_add || ID_addu || ID_sub || ID_subu || ID_or || ID_xor ||
ID_nor || ID_slt || ID_sltu || ID_sll || ID_sllv ||
523. ID_sra || ID_srl || ID_subu || ID_and || ID_sub || ID_srlv ||
ID_srav;
524.
525. reg [4:0]Reg_ID_W_address;
526. always @ (*) begin

```

```

527.     if(ID_add || ID_addu || ID_sub || ID_subu || ID_and || ID_or || ID_xor || ID_nor
|| ID_slt || ID_sltu || ID_sll || ID_srl || ID_sra ||
528.         ID_sllv || ID_srlv || ID_srav)
529.         Reg_ID_W_address <= ID_Rdc;
530.     else if (ID_addi || ID_addiu || ID_andi || ID_ori || ID_xori || ID_lw || ID_slti
|| ID_sltiu || ID_lui )
531.         Reg_ID_W_address <= ID_Rtc;
532.     else if (ID_jal)
533.         Reg_ID_W_address = 5'd31;
534.
535. end
536.
537. wire Reg_MEM_W ;
538. assign Reg_MEM_W = MEM_addi || MEM_addiu || MEM_andi || MEM_ori || MEM_sltiu ||
MEM_lui || MEM_xori || MEM_slti || MEM_sltiu || MEM_lui ||
539.         MEM_add || MEM_addu || MEM_sub || MEM_subu || MEM_or || MEM_xor ||
MEM_nor || MEM_slt || MEM_sltu || MEM_sll || MEM_sllv ||
540.         MEM_sra || MEM_srl || MEM_subu || MEM_and || MEM_sub || MEM_srlv
|| MEM_srav;
541.
542. reg [4:0]Reg_MEM_W_address;
543. always @ (*) begin
544.     if(MEM_add || MEM_addu || MEM_sub || MEM_subu || MEM_and || MEM_or || MEM_xor ||
MEM_nor || MEM_slt || MEM_sltu || MEM_sll || MEM_srl || MEM_sra ||
545.         MEM_sllv || MEM_srlv || MEM_srav)
546.         Reg_MEM_W_address <= MEM_Rdc;
547.     else if (MEM_addi || MEM_addiu || MEM_andi || MEM_ori || MEM_xori || MEM_lw ||
MEM_slti || MEM_sltiu || MEM_lui )
548.         Reg_MEM_W_address <= MEM_Rtc;
549.     else if (MEM_jal)
550.         Reg_MEM_W_address = 5'd31;
551.
552. end
553.
554. wire Reg_EX_W ;
555. assign Reg_EX_W = EX_addi || EX_addiu || EX_andi || EX_ori || EX_sltiu || EX_lui ||
EX_xori || EX_slti || EX_sltiu || EX_lui ||
556.         EX_add || EX_addu || EX_sub || EX_subu || EX_or || EX_xor ||
EX_nor || EX_slt || EX_sltu || EX_sll || EX_sllv ||
557.         EX_sra || EX_srl || EX_subu || EX_and || EX_sub || EX_srlv ||
EX_srav;
558.
559. reg [4:0]Reg_EX_W_address;
560. always @ (*) begin

```

```

561.    if(EX_add || EX_addu || EX_sub || EX_subu || EX_and || EX_or || EX_xor || EX_nor
|| EX_slt || EX_sltu || EX_sll || EX_srl || EX_sra ||
562.        EX_sllv || EX_srlv || EX_srav)
563.        Reg_EX_W_address <= EX_Rdc;
564.    else if (EX_addi || EX_addiu || EX_andi || EX_ori || EX_xori || EX_lw || EX_slti
|| EX_sltiu || EX_lui )
565.        Reg_EX_W_address <= EX_Rtc;
566.    else if (MEM_jal)
567.        Reg_EX_W_address = 5'd31;
568.
569. end
570.
571.
572. wire Reg_ID_RS;
573. wire Reg_ID_RT;
574.
575. assign Reg_ID_RS = ID_addi || ID_addiu || ID_andi || ID_ori || ID_slti || ID_lui ||
ID_xori || ID_sltiu || ID_lui || ID_lw || ID_sw ||
576.        ID_add || ID_addu || ID_sub || ID_subu || ID_and || ID_or || ID_xor
|| ID_nor || ID_slt || ID_sltu || ID_sll || ID_sllv ||
577.        ID_sra || ID_srl || ID_subu || ID_sub || ID_srlv || ID_srav ||
ID_beq || ID_bne || ID_jr ;
578.
579. assign Reg_ID_RT = ID_sw || ID_add || ID_addu || ID_sub || ID_subu || ID_and || ID_or
|| ID_xor || ID_nor
580.        || ID_slt || ID_sltu || ID_sllv ||
581.        ID_subu || ID_sub || ID_srlv || ID_srav || ID_beq || ID_bne ||
ID_bne || ID_srl ;
582.
583. wire [4:0] RT_ID_R_address;
584. wire [4:0] RS_ID_R_address;
585. assign RT_ID_R_address = Reg_ID_RT ? ID_Rtc : 5'hz ;
586. assign RS_ID_R_address = Reg_ID_RS ? ID_Rsc : 5'hz;
587.
588. always @ (*) begin
589.
590. end
591.
592. //跳转指令
593. //IF 段时，如果知道是跳转指令，要看上一指令（即现在在 ID,EX,MEM 段的指令会不会修改跳转指令相
关的寄存器值）
594. wire jump_inst = if_jr || if_bne || if_beq ;
595. //jr 是无条件转移到 rs 中
596.

```



```

597. //halt
598. always @ (*) begin
599.     JUMP_halt = 0 ;
600.     RAW_halt = 0;
601.
602.     if(jump_inst) begin    //跳转指令冲突
603.         if(Reg_ID_W ==1) begin
604.             if(Reg_ID_W_address == if_Rsc) begin
605.
606.                 JUMP_halt = 1;end
607.                 if((if_bne||if_beq) && Reg_ID_W_address == if_Rtc) begin
608.
609.                     JUMP_halt = 1;end
610.             end
611.             if(Reg_EX_W ==1) begin
612.                 if(Reg_EX_W_address == if_Rsc) begin
613.
614.                     JUMP_halt = 1;end
615.                     if((if_bne||if_beq) && Reg_EX_W_address == if_Rtc) begin
616.
617.                         JUMP_halt = 1;end
618.                 end
619.                 if(Reg_MEM_W ==1) begin
620.                     if(Reg_MEM_W_address == if_Rsc) begin
621.
622.                         JUMP_halt = 1;end
623.                         if((if_bne||if_beq) && Reg_MEM_W_address == if_Rtc) begin
624.
625.                             JUMP_halt = 1;end
626.                     end
627.
628.                 end
629.
630.                 if(Reg_EX_W==1) begin    //写读冲突
631.                     if( Reg_EX_W_address == RT_ID_R_address|| Reg_EX_W_address ==
RS_ID_R_address) begin
632.
633.                         RAW_halt=1;end
634.                     end
635.
636.                 if(Reg_MEM_W==1) begin
637.                     if( Reg_MEM_W_address == RT_ID_R_address|| Reg_MEM_W_address ==
RS_ID_R_address) begin
638.

```

```

639.         RAW_halt=1;end
640.     end
641. end
642.
643. wire [31:0] dm_addr;
644. assign dm_addr = ( DM_addr - 32'h1001_0000);
645.
646. my_DMEM my_dmem_inst (
647.     .clk(clk),           // 连接时钟信号
648.     .DM_CS(1'b1),        // 连接 DM_CS 信号
649.     .DM_W(DM_W),         // 连接 DM_W 信号
650.     .DM_R(DM_R),         // 连接 DM_R 信号
651.     .DM_addr(dm_addr),   // 连接 ALU 的输出作为 DMEM 地址
652.     .DM_wdata(DM_wdata), // 连接一些数据信号
653.     .DM_rdata(DM_rdata) // 连接数据输出信号
654. );
655.
656. endmodule
657.

```

```

1. module IMEM(
2.     input [10:0] addr,
3.     output [31:0] output_data
4. );
5.
6. dist_mem_gen_0 dist_mem_gen_0(
7.     .a(addr),
8.     .spo(output_data)
9. );
10.
11. endmodule
12.

```

```

1. module cpu_regfile(
2.     input reg_clk,
3.     input reg_rst, // 复位信号
4.     input reg_ena, // 使能信号 高电平有效
5.     input reg_wena, // 写使能信号 高电平有效
6.     input [4:0] raddr1, // 读的地址 1
7.     input [4:0] raddr2, // 读的地址 2
8.     input [4:0] waddr, // 写地址
9.     input [31:0] wdata, // 写数据
10.    output [31:0] rdata1, // 输出读的数据 1
11.    output [31:0] rdata2 // 输出读的数据 2

```

```
12. );
13.
14. // Regfile 中, Rs 和 Rt 为输出, Rd 为输入。
15. // Rsc, Rtc, Rdc 分别代表读写地址
16.
17. reg [31:0] wyf_array_reg[31:0];
18. integer i;
19.
20. always @(negedge reg_clk or posedge reg_rst) // negedge??
21. begin
22.     if (reg_rst == 1) // 复位信号
23.     begin
24.         wyf_array_reg[0] <= 32'h0;
25.         wyf_array_reg[1] <= 32'h0;
26.         wyf_array_reg[2] <= 32'h0;
27.         wyf_array_reg[3] <= 32'h0;
28.         wyf_array_reg[4] <= 32'h0;
29.         wyf_array_reg[5] <= 32'h0;
30.         wyf_array_reg[6] <= 32'h0;
31.         wyf_array_reg[7] <= 32'h0;
32.         wyf_array_reg[8] <= 32'h0;
33.         wyf_array_reg[9] <= 32'h0;
34.         wyf_array_reg[10] <= 32'h0;
35.         wyf_array_reg[11] <= 32'h0;
36.         wyf_array_reg[12] <= 32'h0;
37.         wyf_array_reg[13] <= 32'h0;
38.         wyf_array_reg[14] <= 32'h0;
39.         wyf_array_reg[15] <= 32'h0;
40.         wyf_array_reg[16] <= 32'h0;
41.         wyf_array_reg[17] <= 32'h0;
42.         wyf_array_reg[18] <= 32'h0;
43.         wyf_array_reg[19] <= 32'h0;
44.         wyf_array_reg[20] <= 32'h0;
45.         wyf_array_reg[21] <= 32'h0;
46.         wyf_array_reg[22] <= 32'h0;
47.         wyf_array_reg[23] <= 32'h0;
48.         wyf_array_reg[24] <= 32'h0;
49.         wyf_array_reg[25] <= 32'h0;
50.         wyf_array_reg[26] <= 32'h0;
51.         wyf_array_reg[27] <= 32'h0;
52.         wyf_array_reg[28] <= 32'h0;
53.         wyf_array_reg[29] <= 32'h0;
54.         wyf_array_reg[30] <= 32'h0;
55.         wyf_array_reg[31] <= 32'h0;
```

```

56. end
57. else if (reg_ena == 1 & reg_wena == 1 & (waddr != 5'b00000)) // 如果写信号为高电平,
    并且写的 address 不是 0, 那么就写
58. begin
59.     wyf_array_reg[waddr] <= wdata;
60. end
61. end
62.
63. assign rdata1 = (reg_ena == 1) ? wyf_array_reg[raddr1] : 32'hzzzzzzzz;
64. assign rdata2 = (reg_ena == 1) ? wyf_array_reg[raddr2] : 32'hzzzzzzzz;
65.
66. endmodule
67.

```

3、EX_MEM

```

1. module EX_MEM (
2.     input wire clk,
3.     input wire rst,
4.     input wire [31:0] ExAluOut,
5.     input wire [31:0] IdExPassData,
6.     input wire [31:0] IdExIr,
7.     input wire overflow,
8.     input wire [31:0] IdExAlub,
9.
10.    output reg [31:0] ExMeAluOut,
11.    output reg [31:0] ExMePassData,
12.    output reg [31:0] ExMeIr,
13.    output reg ExMeOverflowFlag,
14.    output reg [31:0] ExMeAlub
15. );
16.
17. always @(posedge clk or posedge rst) begin
18.     if (rst) begin
19.         ExMeAluOut <= 32'b00000000;
20.         ExMePassData <= 32'b00000000;
21.         ExMeIr <= 32'hffffffff;
22.         ExMeOverflowFlag <= 1'b00000000;
23.         ExMeAlub <= 32'b00000000;
24.     end
25.     else begin
26.         ExMeAluOut <= ExAluOut;
27.         ExMePassData <= IdExPassData;
28.         ExMeIr <= IdExIr;
29.         ExMeOverflowFlag <= overflow;

```

```

30.     ExMeAlub <= IdExAlub; // Update to use IdExAlub as input
31. end
32. end
33.
34. endmodule

```

4、ID_EX

```

1. module ID_EX(
2.   input wire clk,
3.   input wire rst,
4.   input wire RAW_halt,
5.   input wire [31:0] id_mux_alua_out,
6.   input wire [31:0] id_mux_alub_out,
7.   input wire [31:0] id_pass_data,
8.   input wire [31:0] ifid_ir,
9.   output reg [31:0] IdExAluA,
10.  output reg [31:0] IdExAluB,
11.  output reg [31:0] IdEx_transfer,
12.  output reg [31:0] IdEx_Ir
13. );
14.
15. always @(posedge clk or posedge rst) begin
16.   if (rst || RAW_halt ) begin
17.     IdExAluA <= 32'b0;
18.     IdExAluB <= 32'b0;
19.     IdEx_transfer <= 32'b0;
20.     IdEx_Ir <= 32'hffffffff;
21.   end
22.
23.   else begin
24.     IdExAluA <= id_mux_alua_out;
25.     IdExAluB <= id_mux_alub_out;
26.     IdEx_transfer <= id_pass_data;
27.     IdEx_Ir <= ifid_ir;
28.   end
29. end
30.
31. endmodule
32.

```

5、IF_Decoder

```

1. `include "def.vh"
2.

```

```

3. module IF_Decoder(
4.   input wire [31:0] if_ir,    // 输入指令
5.   output wire [5:0]if_op,
6.   output wire [4:0]if_shamt,
7.   output wire [5:0]if_func,
8.   output wire [15:0]if_imm16,
9.   output wire [4:0]if_Rsc,
10.  output wire [4:0]if_Rtc,
11.  output wire [4:0]if_Rdc,
12.  output wire if_add,          // ADD 指令
13.  output wire if_addu,         // ADDU 指令
14.  output wire if_sub,          // SUB 指令
15.  output wire if_subu,         // SUBU 指令
16.  output wire if_and,          // AND 指令
17.  output wire if_or,           // OR 指令
18.  output wire if_xor,          // XOR 指令
19.  output wire if_nor,          // NOR 指令
20.  output wire if_slt,          // SLT 指令
21.  output wire if_sltu,         // SLTU 指令
22.  output wire if_sll,          // SLL 指令
23.  output wire if_srl,          // SRL 指令
24.  output wire if_sra,          // SRA 指令
25.  output wire if_sllv,         // SLLV 指令
26.  output wire if_srlv,         // SRLV 指令
27.  output wire if_srav,         // SRAV 指令
28.  output wire if_jr,           // JR 指令
29.  output wire if_addi,         // ADDI 指令
30.  output wire if_addiu,        // ADDIU 指令
31.  output wire if_andi,         // ANDI 指令
32.  output wire if_ori,          // ORI 指令
33.  output wire if_xori,         // XORI 指令
34.  output wire if_lw,           // LW 指令
35.  output wire if_sw,           // SW 指令
36.  output wire if_beq,          // BEQ 指令
37.  output wire if_bne,          // BNE 指令
38.  output wire if_slti,         // SLTI 指令
39.  output wire if_sltiu,        // SLTIU 指令
40.  output wire if_lui,          // LUI 指令
41.  output wire if_j,            // J 指令
42.  output wire if_jal           // JAL 指令
43.  );
44.
45. assign if_op = if_ir[31:26] ;
46. assign if_shamt = if_ir[10:6];

```

```

47. assign if_func = if_ir[5:0];
48. assign if_imm16 = if_ir[15:0];
49. assign if_Rsc = if_ir[25:21];
50. assign if_Rtc = if_ir[20:16];
51. assign if_Rdc = if_ir[15:11];
52.
53. `define IS_OP_COMMON(op) (if_op == `OP_COMMON && if_func == op)
54. `define IS_OP(op) (if_op == op)
55.
56. assign if_add = `IS_OP_COMMON(`FUNCT_ADD); // ADD 指令
57. assign if_addu = `IS_OP_COMMON(`FUNCT_ADDU); // ADDU 指令
58. assign if_sub = `IS_OP_COMMON(`FUNCT_SUB); // SUB 指令
59. assign if_subu = `IS_OP_COMMON(`FUNCT_SUBU); // SUBU 指令
60. assign if_and = `IS_OP_COMMON(`FUNCT_AND); // AND 指令
61. assign if_or = `IS_OP_COMMON(`FUNCT_OR); // OR 指令
62. assign if_xor = `IS_OP_COMMON(`FUNCT_XOR); // XOR 指令
63. assign if_nor = `IS_OP_COMMON(`FUNCT_NOR); // NOR 指令
64. assign if_slt = `IS_OP_COMMON(`FUNCT_SLT); // SLT 指令
65. assign if_sltu = `IS_OP_COMMON(`FUNCT_SLTU); // SLTU 指令
66. assign if_sll = `IS_OP_COMMON(`FUNCT_SLL); // SLL 指令
67. assign if_srl = `IS_OP_COMMON(`FUNCT_SRL); // SRL 指令
68. assign if_sra = `IS_OP_COMMON(`FUNCT_SRA); // SRA 指令
69. assign if_sllv = `IS_OP_COMMON(`FUNCT_SLLV); // SLLV 指令
70. assign if_srlv = `IS_OP_COMMON(`FUNCT_SRLV); // SRLV 指令
71. assign if_srav = `IS_OP_COMMON(`FUNCT_SRAV); // SRAV 指令
72. assign if_jr = `IS_OP_COMMON(`FUNCT_JR); // JR 指令
73.
74. // Immediate 指令
75. assign if_addi = `IS_OP(`OP_ADDI); // ADDI 指令
76. assign if_addiu = `IS_OP(`OP_ADDIU); // ADDIU 指令
77. assign if_andi = `IS_OP(`OP_ANDI); // ANDI 指令
78. assign if_ori = `IS_OP(`OP_ORI); // ORI 指令
79. assign if_xori = `IS_OP(`OP_XORI); // XORI 指令
80. assign if_lw = `IS_OP(`OP_LW); // LW 指令
81. assign if_sw = `IS_OP(`OP_SW); // SW 指令
82. assign if_beq = `IS_OP(`OP_BEQ); // BEQ 指令
83. assign if_bne = `IS_OP(`OP_BNE); // BNE 指令
84. assign if_slti = `IS_OP(`OP_SLTI); // SLTI 指令
85. assign if_sltiu = `IS_OP(`OP_SLTIU); // SLTIU 指令
86. assign if_lui = `IS_OP(`OP_LUI); // LUI 指令
87.
88. // Jump 指令
89. assign if_j = `IS_OP(`OP_J); // J 指令
90. assign if_jal = `IS_OP(`OP_JAL); // JAL 指令

```

```
91.  
92. endmodule  
93.
```

5、 IF_ID

```
1. module IF_ID(  
2.   input wire clk,  
3.   input wire rst,  
4.   input wire JUMP_halt,  
5.   input wire RAW_halt,  
6.   input wire [31:0] If_instruction,  
7.   input wire [31:0] If_PC,  
8.   output reg [31:0] IfId_Npc,  
9.   output reg [31:0] IfId_Ir  
10. );  
11.  
12. always @(posedge clk or posedge rst) begin  
13.   if (rst) begin  
14.     IfId_Ir <= 32'hffffffff;  
15.     IfId_Npc <= 32'b0;  
16.   end  
17.   else if (JUMP_halt == 1 && RAW_halt == 0) begin  
18.     IfId_Ir <= 32'hffffffff;  
19.   end  
20.   else if (JUMP_halt == 0 && RAW_halt == 0) begin  
21.     IfId_Npc <= If_PC + 32'h4;  
22.     IfId_Ir <= If_instruction;  
23.   end  
24. end  
25.  
26. endmodule  
27.
```

5、 MEM_WB

```
1. module MEM_WB (  
2.   input wire clk,  
3.   input wire rst,  
4.   input wire MeLw,  
5.   input wire [31:0] MeDmemOut,  
6.   input wire [31:0] ExMeAluOut,  
7.   input wire [31:0] ExMePassData,  
8.   input wire [31:0] ExMeIr,  
9.   input wire ExMeOverflowFlag,  
10.  input wire [31:0] ExMeAlub,
```



```

11.
12.  output reg [31:0] MemWbAluOut,
13.  output reg [31:0] MeWbPassData,
14.  output reg [31:0] MemWbIr,
15.  output reg MemWbOverflowFlag,
16.  output reg [31:0] MemWbAlub
17. );
18.
19. always @ (posedge clk or posedge rst) begin
20.     if (rst) begin
21.         MemWbAluOut <= 32'b0000;
22.         MemWbOverflowFlag <= 32'b00000;
23.         MemWbAlub <= 32'b00000;
24.         MemWbIr <= 32'hffffffff;
25.         MeWbPassData <= 32'b00000;
26.     end
27.     else begin
28.         MemWbAluOut <= ExMeAluOut;
29.         MemWbIr <= ExMeIr;
30.         MemWbOverflowFlag <= ExMeOverflowFlag;
31.         MemWbAlub <= ExMeAlub;
32.         if (MeLw)
33.             MeWbPassData <= MeDmemOut;
34.         else
35.             MeWbPassData <= ExMePassData;
36.     end
37. end
38.
39. endmodule
40.

```

6、 my_ALU

```

1. module my_ALU(
2.     input [31:0] operator_a,      //操作数 a
3.     input [31:0] operator_b,      //操作数 b
4.     input [3:0] alu_controller,   //alu3210
5.     output reg [31:0] result,      //计算结果
6.
7.     output reg zero,              //结果为 0
8.     output reg carry,             //进位
9.     output reg negative,          //sub 的结果为负数
10.    output reg overflow            //溢出
11. );
12.    reg [32:0] r_temp;

```

```

13.    wire signed [31:0] s_a;
14.    wire signed[31:0] s_b;
15.    assign s_a=operator_a;
16.    assign s_b=operator_b;
17.
18.    always @ (*) begin
19.        case(alu_controller)
20.            4'b0000:
21.                //Addu lw,sw 无符号加法
22.                begin
23.                    r_temp<=operator_a+operator_b;
24.                    result=r_temp[31:0];
25.                    /*四个标志位*/
26.                    carry=r_temp[32];
27.                    zero=(r_temp==32'b0)?1:0;
28.                    negative=0;
29.                    overflow=0;
30.                    // negative=r_temp[31]; //是否是负数看符号位
31.                    // overflow
32.                end
33.            4'b0001:
34.                //subu 无符号减法
35.                begin
36.                    r_temp<=operator_a-operator_b;
37.                    result=r_temp[31:0];
38.                    /*四个标志位*/
39.                    if(operator_a<operator_b)
40.                        carry=1;
41.                    else
42.                        carry=0;
43.
44.                    zero=(r_temp==32'b0)?1:0;
45.                    negative=0;
46.                    overflow=0;
47.                end
48.
49.            4'b0010:
50.                //add 有符号数加法
51.                begin
52.                    r_temp<=s_a+s_b;
53.                    result=r_temp[31:0];
54.                    /*四个标志位*/
55.                    //if(operator_a<operator_b)
56.                    carry=1;

```

```

57.         //else
58.             //carry<=0;
59.             carry=r_temp[32]; //
60.
61.             zero=(r_temp==32'b0)?1:0; //
62.             negative=r_temp[31]; //
63.
64.             //if(r_temp[32] != r_temp[31] && operator_a[31] == operator_b[31])
65.                 //overflow = 1;
66.             //else
67.                 //overflow = 0;
68.             overflow=r_temp[31]; //
69.         end
70.
71.     4'b0011:
72.     //sub 有符号数减法
73.         begin
74.             r_temp<=s_a-s_b;
75.             result=r_temp[31:0];
76.             /*四个标志位*/
77.             carry=r_temp[32]; //
78.             zero=(r_temp==32'b0)?1:0; //
79.             negative=r_temp[31]; //
80.             overflow=r_temp[31]; //
81.         end
82.
83.     4'b0100:
84.     //and 按位且 用无符号数
85.         begin
86.             r_temp<=operator_a & operator_b;
87.             result=r_temp[31:0];
88.             /*四个标志位*/
89.             carry=r_temp[32]; //
90.             zero=(r_temp==32'b0)?1:0; //
91.             negative=r_temp[31]; //
92.             overflow=r_temp[31]; //
93.         end
94.
95.     4'b0101:
96.     //or 按位或
97.         begin
98.             r_temp<=operator_a | operator_b;
99.             result<=r_temp[31:0];
100.            /*四个标志位*/

```

```

101.         carry=r_temp[32]; //
102.         zero=(r_temp==32'b0)?1:0; //
103.         negative=r_temp[31]; //
104.         overflow=r_temp[31];    //
105.     end
106.
107. 4'b0110:
108.     //xor 异或
109.     begin
110.         r_temp<=operator_a ^ operator_b;
111.         result<=r_temp[31:0];
112.         /*四个标志位*/
113.         carry=r_temp[32]; //
114.         zero=(r_temp==32'b0)?1:0; //
115.         negative=r_temp[31]; //
116.         overflow=r_temp[31];    //
117.     end
118.
119. 4'b0111:
120.     //nor 或非
121.     begin
122.         r_temp<=~(operator_a | operator_b);
123.         result=r_temp[31:0];
124.         /*四个标志位*/
125.         carry=r_temp[32]; //
126.         zero=(r_temp==32'b0)?1:0; //
127.         negative=r_temp[31]; //
128.         overflow=r_temp[31];    //
129.     end
130.
131. 4'b1000:  //???为什么有两个 lui
132.     //lui 腾空低 16 位
133.     begin
134.         r_temp<={operator_b[15:0], 16'b0};
135.         result=r_temp[31:0];
136.         /*四个标志位*/
137.         carry=r_temp[32]; //
138.         zero=(r_temp==32'b0)?1:0; //
139.         negative=r_temp[31]; //
140.         overflow=r_temp[31];    //
141.     end
142. 4'b1001:  //???为什么有两个 lui
143.         //lui 腾空低 16 位
144.         begin

```

```

145.             r_temp<={operator_b[15:0], 16'b0};
146.             result=r_temp[31:0];
147.             /*四个标志位*/
148.             carry=r_temp[32]; //
149.             zero=(r_temp==32'b0)?1:0; //
150.             negative=r_temp[31]; //
151.             overflow=r_temp[31];    //
152.         end
153. 4'b1010:
154.     //sltu 无符号比大小
155.     begin
156.         r_temp<=operator_a < operator_b ? 1:0 ;
157.         result=r_temp[31:0];
158.         /*四个标志位*/
159.         carry=r_temp[32]; //
160.         zero=(r_temp==32'b0)?1:0; //
161.         negative=r_temp[31]; //
162.         overflow=r_temp[31];    //
163.     end
164. 4'b1011:
165.     //slt 比较大小
166.     begin
167.         r_temp <= s_a < s_b ? 1:0 ;
168.         result = r_temp[31:0];
169.         /*四个标志位*/
170.         carry=r_temp[32]; //
171.         zero=(r_temp==32'b0)?1:0; //
172.         negative=r_temp[31]; //
173.         overflow=r_temp[31];    //
174.
175.     end
176. 4'b1100:
177.     //sra sra 算术右移
178.     begin
179.         r_temp <= s_b >>> s_a;
180.         result = r_temp[31:0];
181.         /*四个标志位*/
182.         carry=r_temp[32]; //
183.         zero=(r_temp==32'b0)?1:0; //
184.         negative=r_temp[31]; //
185.         overflow=r_temp[31];    //
186.     end
187.
188. 4'b1101:

```

```

189.         //srl , srlv  逻辑右移
190.         begin
191.             r_temp <= operator_b >> operator_a;
192.             result = r_temp[31:0];
193.             /*四个标志位*/
194.             carry=r_temp[32]; //
195.             zero=(r_temp==32'b0)?1:0; //
196.             negative=r_temp[31]; //
197.             overflow=r_temp[31]; //
198.         end
199.     4'b1110:
200.         //sll sllv 逻辑左移
201.         begin
202.             r_temp <= operator_b << operator_a;
203.             result = r_temp[31:0];
204.             /*四个标志位*/
205.             carry =r_temp[32]; //
206.             zero =(r_temp==32'b0)?1:0; //
207.             negative =r_temp[31]; //
208.             overflow =r_temp[31]; //
209.         end
210.     endcase
211. end
212.
213. endmodule
214.

```

7、 my_DMEN

```

1. module my_DMEN(
2.     input clk,
3.     input DM_CS,
4.     input DM_W,
5.     input DM_R,
6.     input [10:0] DM_addr, //ALU 的 result 作为输入
7.     input [31:0] DM_wdata,
8.     output [31:0] DM_rdata //DMEM 的输出
9. );
10.
11.     reg [31:0] DMEM_dataset[31:0];
12.     assign DM_rdata = (DM_R && DM_CS && !DM_W) ? DMEM_dataset[DM_addr] : 32'bz;
13.
14.     always @(negedge clk ) //此处的时序需要再考虑
15.     //always@( posedge clk or negedge DM_CS)
16.     begin

```

```

17.         if (DM_W && DM_CS && !DM_R) begin
18.             DMEM_dataset[DM_addr] <= DM_wdata;
19.         end
20.     end
21.
22.
23. endmodule
24.

```

8、ThreeInputSelector

```

1. module ThreeInputSelector (
2.     input [31:0] data0,
3.     input [31:0] data1,
4.     input [31:0] data2,
5.     input [2:0] selector,
6.     output reg [31:0] output_data
7. );
8.
9. always @*
10. begin
11.     case (selector)
12.         3'b001: output_data = data0;
13.         3'b010: output_data = data1;
14.         3'b100: output_data = data2;
15.
16.         default: output_data = 32'hffffffff; // 默认情况下输出全 1
17.     endcase
18. end
19.
20. endmodule
21.

```

9、pc_reg

```

1. `timescale 1ns / 1ps
2. module pc_reg(
3.     input clk,
4.     input rst,
5.     input ena,
6.     input [31:0] in_data,
7.     output [31:0] out_data
8. );
9.     reg [31:0] temp_out;
10.
11.     always@(negedge clk or posedge rst) begin

```

```
12.  
13.     if(ena)  
14.         temp_out <= in_data;  
15.     else if(rst==1)  
16.         temp_out <= 32'h0040_0000;  
17.     end  
18.  
19.     assign out_data = ena?temp_out:32'hz;  
20. endmodule
```